

计算机体系结构

L03 复杂流水线

计算机体系结构

背景知识 (自学) ③

存储层次 ④

单处理器 (指令级并行) ④

多处理器 (线程级并行) ⑤

加速器 (数据并行) ③

简单流水线 ④

复杂流水线

多周期操作

相关性问题

记分板

基本概念

寄存器重命名

Tomasulo 算法

精确异常

乱序执行

访存操作

Load Bypassing

Load Forwarding

乱序发射

分支预测 ③

指令并行高级技术 ②

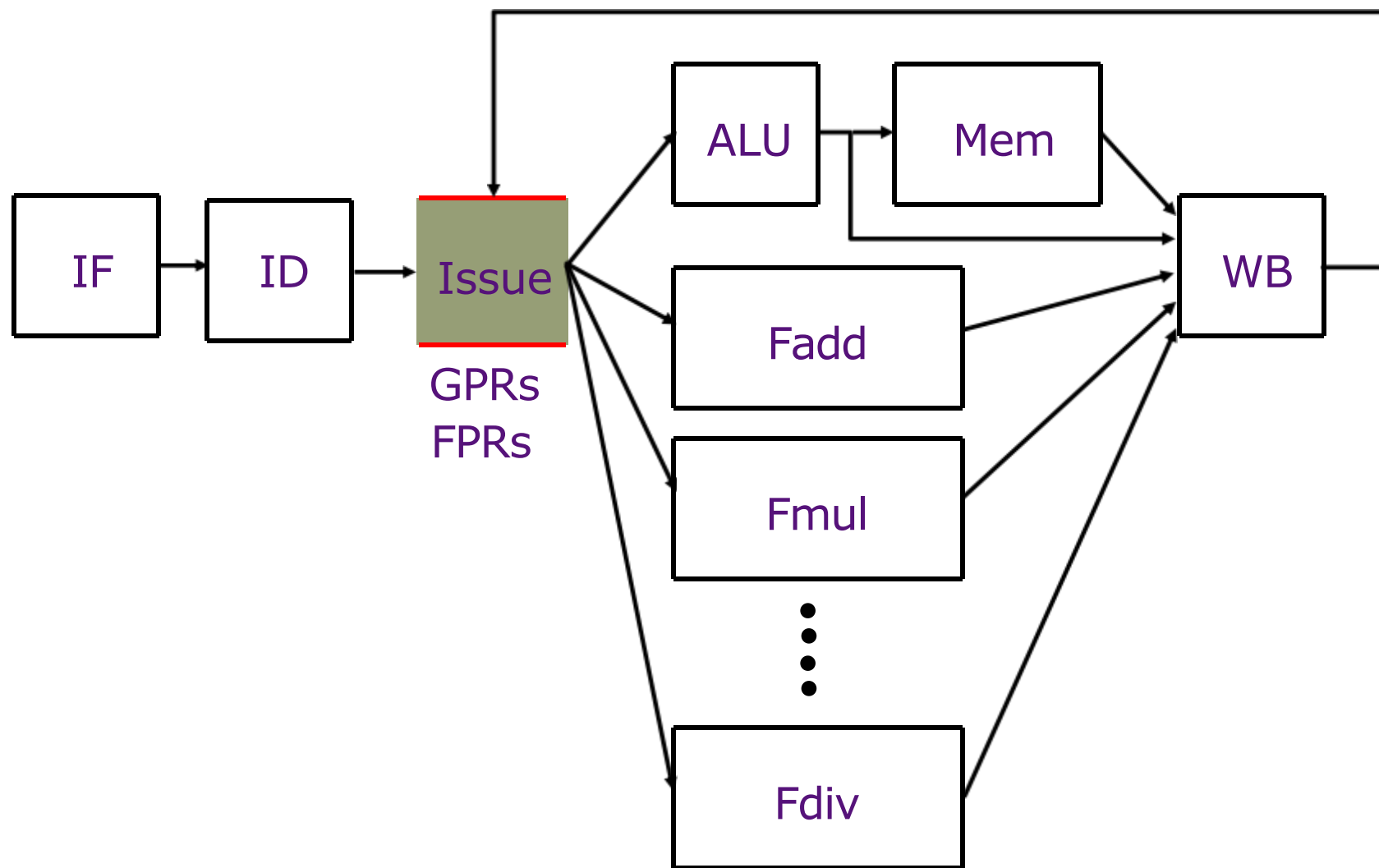
多周期操作

复杂流水线：动机

□ 考虑下面几种情况

- ✓ 多周期操作
 - 例如，浮点运算，乘除法等
- ✓ 延迟不确定操作
 - 内存访问
- ✓ 多个（重复的）功能单元
 - 多个浮点运算单元

复杂流水线结构



多周期操作引发的问题

□ **结构冲突**：多条指令争抢同一个单元

- 例如，同时写回寄存器

□ **数据相关性**：指令不按顺序写回，导致数据相关性问题

多周期操作引发的结构冲突

□ 结构冲突

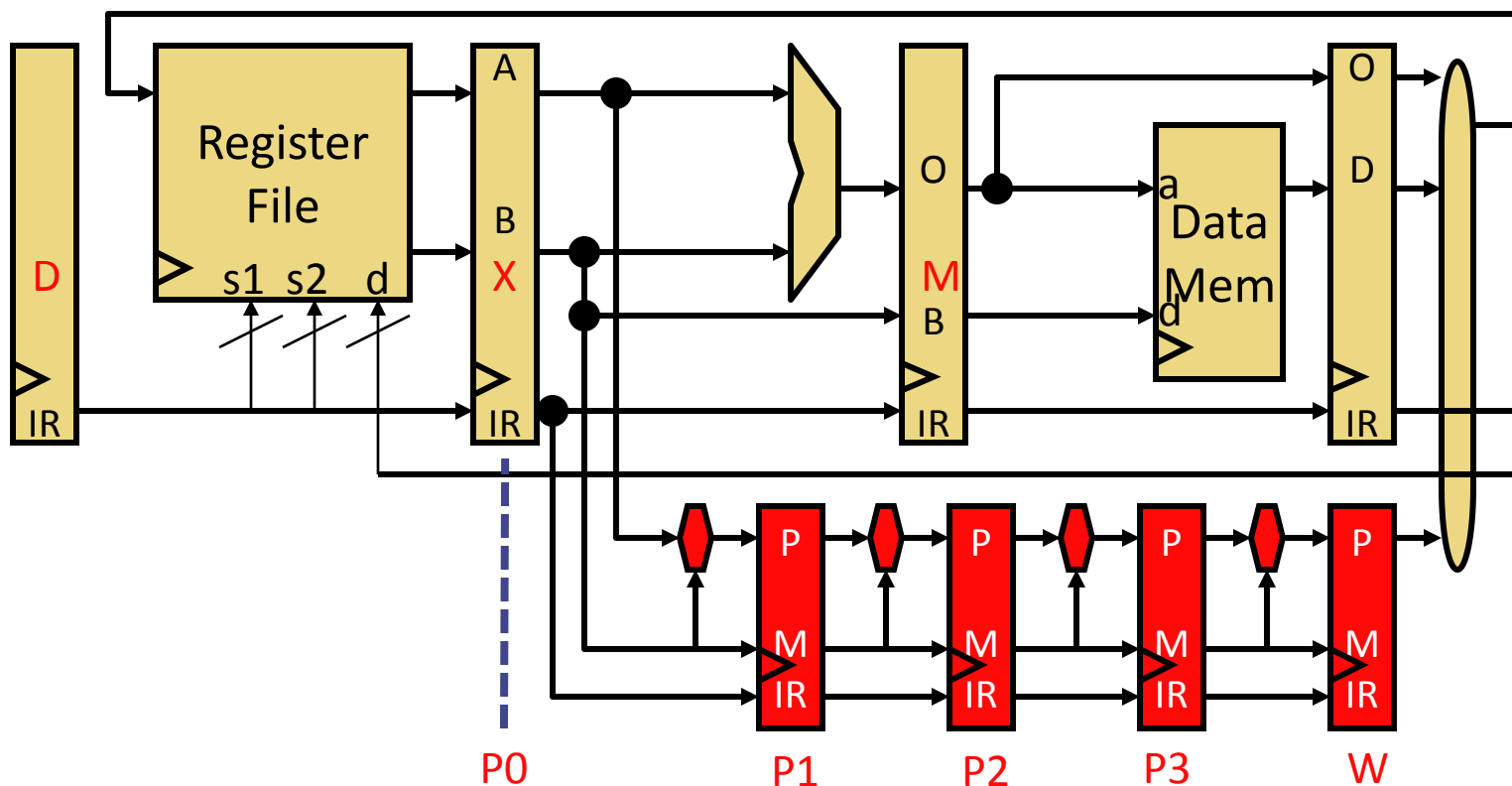
- 两条指令在同一个周期写同一个寄存器文件（通常寄存器文件只有一个写端口）

	1	2	3	4	5	6	7	8	9
mul \$4,\$3,\$5	F	D	P0	P1	P2	P3	W		
addi \$6,\$1,1		F	D	X	M	W			
add \$5,\$6,\$10			F	D	X	M	W		

- 解决方案? stall后续指令

	1	2	3	4	5	6	7	8	9
mul \$4,\$3,\$5	F	D	P0	P1	P2	P3	W		
addi \$6,\$1,1		F	D	X	M	W			
add \$5,\$6,\$10			F	D	d*	X	M	W	

避免结构冲突



- **解决前面例子中的结构冲突:**

```
Stall = (OldStallLogic) ||
```

**(D.IR.RegDest "is valid" &&
D.IR.Operation != MULT && P1.IR.RegDest "is valid")**

指令不按顺序写回引发的数据相关性问题

□ **数据相关性**：指令不按顺序写回，导致数据相关性问题

	1	2	3	4	5	6	7	8	9
mul \$4 , \$3, \$5	F	D	P0	P1	P2	P3	W		
addi \$4 , \$1, 1		F	D	X	M	W			
...									
...									
add \$10, \$4 , \$6					F	D	X	M	W

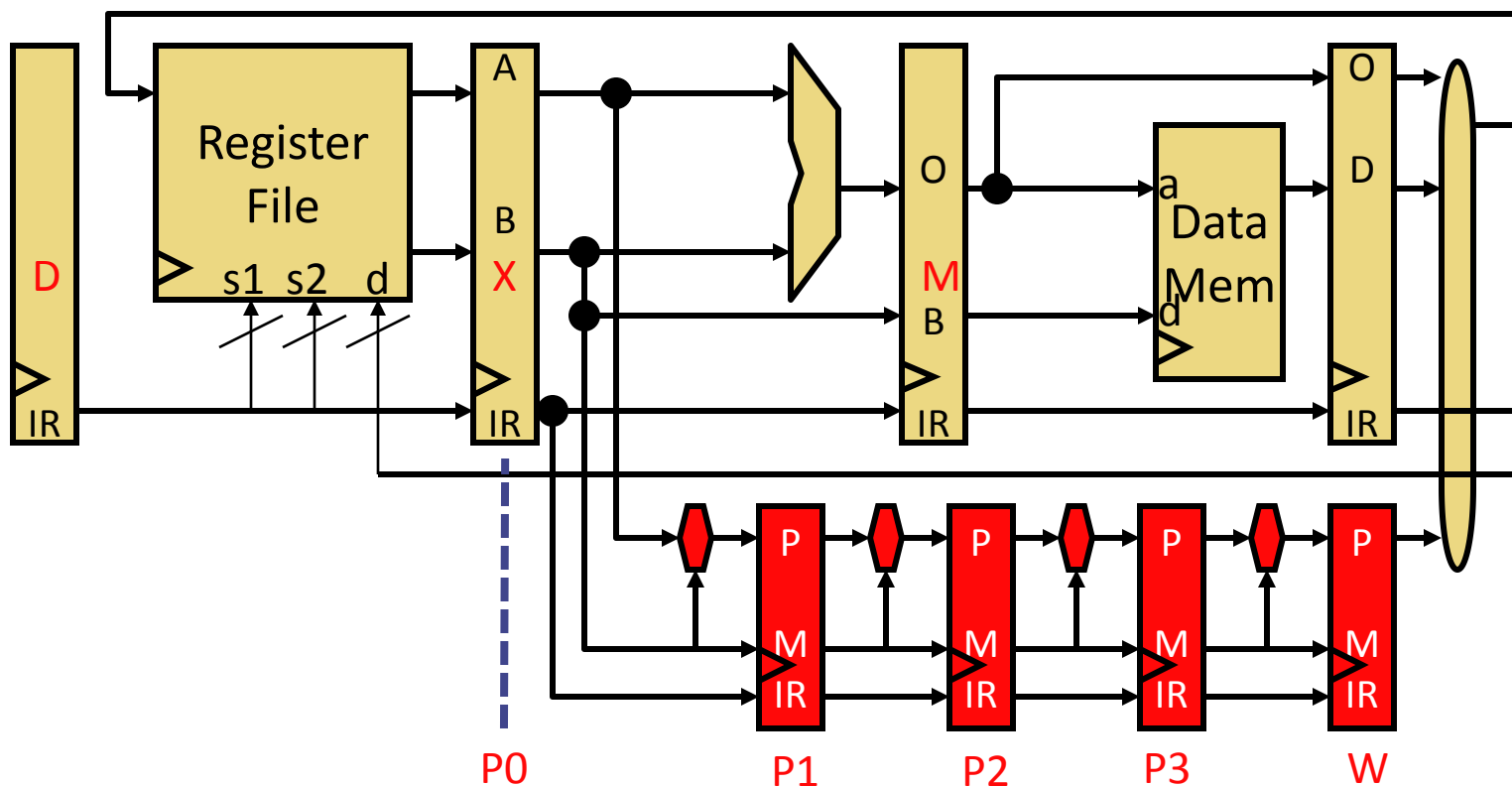
add指令读取的\$4应该是addi指令的结果，但实际上是mul的结果

避免乱序写寄存器

□ addi指令增加两个周期的stall

	1	2	3	4	5	6	7	8	9
mul \$4 , \$3, \$5	F	D	P0	P1	P2	P3	W		
addi \$4 , \$1, 1		F	D	d*	d*	X	M	W	
...									
...									
add \$10, \$4 , \$6					F	D	X	M	W

避免乱序写寄存器



- 如果发现, **stall**:

$\text{Stall} = (\text{OldStallLogic}) \parallel$

$((\text{D.IR.RegDest} == \text{P0.IR.RegDest}) \parallel$
 $(\text{D.IR.RegDest} == \text{P1.IR.RegDest}))$

记分板

(Scoreboard)

多周期操作问题的解决方案 - 记分板

□ 增加**指令发射阶段(issue)**

- ✓ decode完的指令尝试发射到执行单元
- ✓ **发射条件：**
 - 指令需要的功能单元空闲
 - 操作数已经准备好 → 解决RAW
 - 将要写的目标寄存器安全 → 解决WAW
 - 如果前面指令和该指令目标寄存器相同，不发射
 - 写回阶段不存在结构冲突
 - 如果前面指令和该指令目标寄存器相同，不会发射
- ✓ 指令按顺序发射，前面指令不发射，后面不能前进
 - 不存在WAR问题
 - load指令发送的时候，操作数已经准备好

记分板硬件实现

□ 每个功能单元维护一个状态标志位 **Busy**

- 为**1**表示功能单元正忙，**0**表示空闲

	FU1	FU2	FU3	FU4	FU5
Busy					

□ 每个寄存器维护一个写回标记位 **WP**

- 为**1**表示有指令要写该寄存器，**0**表示没有

	F1	F2	F3	F4	F5	F6	F7	F8
WP								

记分板示例

[illegible]
$$I_1$$
 I_2
$$I_3$$
$$I_4$$
$$I_5$$

I_6

DIVD

LD

MULTD

DIVD

SUBD

ADDD

f6,

f2,

f0,

f8,

f10,

f6,

f6,

45(r3)

f2,

f6,

f0,

f8,

f4

f4

f2

f6

f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 <i>I₁</i>						f6					f6

<i>I</i> ₁	DIVD	f6,	f6,	f4
<i>I</i> ₂	LD	f2,	45(r3)	
<i>I</i> ₃	MULTD	f0,	f2,	f4
<i>I</i> ₄	DIVD	f8,	f6,	f2
<i>I</i> ₅	SUBD	f10,	f0,	f6
<i>I</i> ₆	ADDD	f6,	f8,	f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 I₁						f6					f6
t1 I₂	f2						f6				f6, f2

<i>I</i> ₁	DIVD	f6,	f6,	f4
<i>I</i> ₂	LD	f2,	45(r3)	
<i>I</i> ₃	MULTD	f0,	f2,	f4
<i>I</i> ₄	DIVD	f8,	f6,	f2
<i>I</i> ₅	SUBD	f10,	f0,	f6
<i>I</i> ₆	ADDD	f6,	f8,	f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 <i>I</i>₁						f6					f6
t1 <i>I</i>₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 <i>I</i>₂

*I*₁

*I*₂

*I*₃

*I*₄

*I*₅

*I*₆

DIVD

LD

MULTD

DIVD

SUBD

ADDD

f6,

f2,

f0,

f8,

f10,

f6,

f6,

45(r3)

f2,

f6,

f0,

f8,

f4

f4

f2

f6

f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 I₁						f6					f6
t1 I₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 I₂
t3 I₃			f0						f6		f6, f0

I₁	DIVD	f6,	f6,	f4
I₂	LD	f2,	45(r3)	
I₃	MULTD	f0,	f2,	f4
I₄	DIVD	f8,	f6,	f2
I₅	SUBD	f10,	f0,	f6
I₆	ADDD	f6,	f8,	f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 I₁						f6					f6
t1 I₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 I₂
t3 I₃			f0						f6		f6, f0
t4				f0						f6	f6, f0 I₁

I₁

I₂

I₃

I₄

I₅

I₆

DIVD

LD

MULTD

DIVD

SUBD

ADDD

f6,

f2,

f0,

f8,

f10,

f6,

f6,

45(r3)

f2,

f6,

f0,

f8,

f4

f4

f2

f6

f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 I₁						f6					f6
t1 I₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 I₂
t3 I₃			f0						f6		f6, f0
t4				f0						f6	f6, f0 I₁
t5 I₄					f0	f8					f0, f8

<i>I</i> ₁	DIVD	f6,	f6,	f4
<i>I</i> ₂	LD	f2,	45(r3)	
<i>I</i> ₃	MULTD	f0,	f2,	f4
<i>I</i> ₄	DIVD	f8,	f6,	f2
<i>I</i> ₅	SUBD	f10,	f0,	f6
<i>I</i> ₆	ADDD	f6,	f8,	f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 I₁						f6					f6
t1 I₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 I₂
t3 I₃			f0						f6		f6, f0
t4				f0						f6	f6, f0 I₁
t5 I₄					f0	f8					f0, f8
t6							f8			f0	f0, f8 I₃

<i>I</i> ₁	DIVD	f6,	f6,	f4
<i>I</i> ₂	LD	f2,	45(r3)	
<i>I</i> ₃	MULTD	f0,	f2,	f4
<i>I</i> ₄	DIVD	f8,	f6,	f2
<i>I</i> ₅	SUBD	f10,	f0,	f6
<i>I</i> ₆	ADDD	f6,	f8,	f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 <i>I</i> ₁						f6					f6
t1 <i>I</i> ₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 <i>I</i> ₂
t3 <i>I</i> ₃			f0						f6		f6, f0
t4				f0						f6	f6, f0 <i>I</i> ₁
t5 <i>I</i> ₄					f0	f8					f0, f8
t6							f8			f0	f0, f8 <i>I</i> ₃
t7 <i>I</i> ₅		f10						f8			f8, f10

*I*₁
*I*₂
*I*₃
*I*₄
*I*₅
*I*₆

DIVD
LD
MULTD
DIVD
SUBD
ADDD

f6,
f2,
f0,
f8,
f10,
f6,

f6,
45(r3)
f2,
f6,
f0,
f8,

f4
f4
f2
f6
f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 I₁						f6					f6
t1 I₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 I₂
t3 I₃			f0						f6		f6, f0
t4				f0						f6	f6, f0 I₁
t5 I₄					f0	f8					f0, f8
t6							f8			f0	f0, f8 I₃
t7 I₅		f10						f8			f8, f10
t8									f8	f10	f8, f10 I₅

*I*₁
*I*₂
*I*₃
*I*₄
*I*₅
*I*₆

DIVD
LD
MULTD
DIVD
SUBD
ADDD

f6,
f2,
f0,
f8,
f10,
f6,

f6,
45(r3)
f2,
f6,
f0,
f8,

f4
f4
f2
f6
f2

记分板示例

	功能单元									WB	写回标记位(WP)为1 的寄存器
	Int(1)	Add(1)	Mult(3)			Div(4)					
t0 <i>I</i>₁						f6					f6
t1 <i>I</i>₂	f2						f6				f6, f2
t2								f6		f2	f6, f2 <i>I</i>₂
t3 <i>I</i>₃			f0						f6		f6, f0
t4				f0						f6	f6, f0 <i>I</i>₁
t5 <i>I</i>₄					f0	f8					f0, f8
t6							f8			f0	f0, f8 <i>I</i>₃
t7 <i>I</i>₅		f10						f8			f8, f10
t8									f8	f10	f8, f10 <i>I</i>₅
t9										f8	f8, <i>I</i>₄

I₁
I₂
I₃
I₄
I₅
I₆

DIVD
LD
MULTD
DIVD
SUBD
ADDD

f6,
f2,
f0,
f8,
f10,
f6,

f6,
45(r3)
f2,
f6,
f0,
f8,

f4

f4
f2
f6
f2

截至目前，流水线：

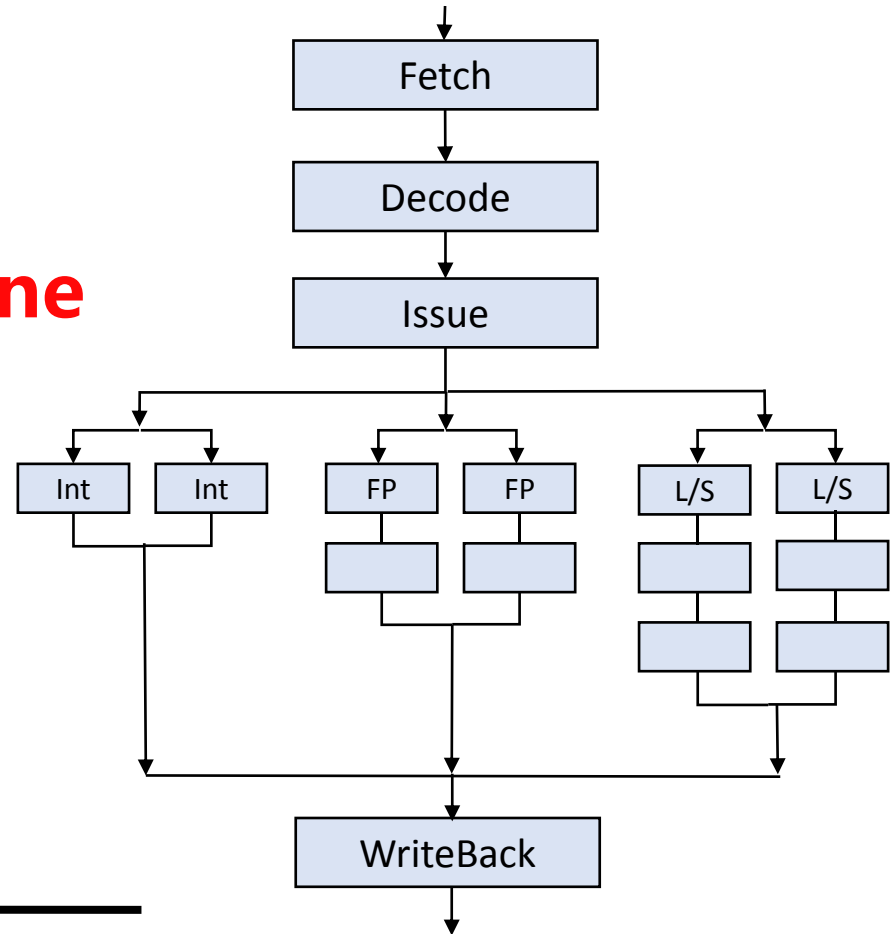
➤ Scalar pipeline

➤ In-order (issue) pipeline

~~➤ 指令无数据相关性~~

➤ 无分支指令

~~➤ 所有指令单个周期完成~~



乱序执行 (动机)

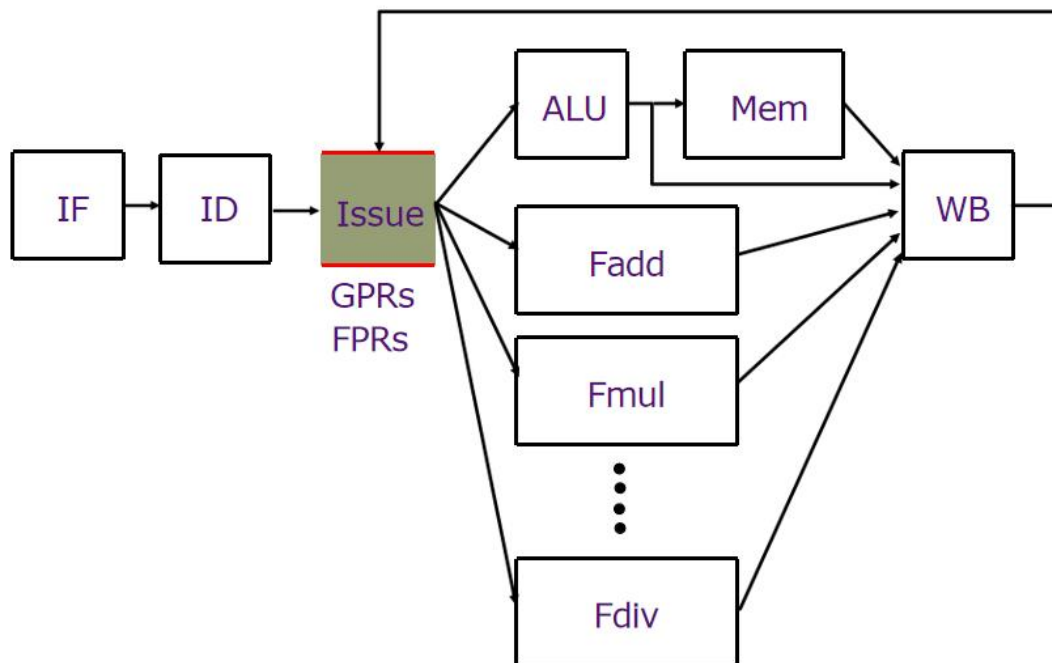
记分板的缺点

□ 指令按顺序发送，当以下条件满足时，指令才能发送

- 不会引起 RAW hazard
- 不会引起 WAW hazard

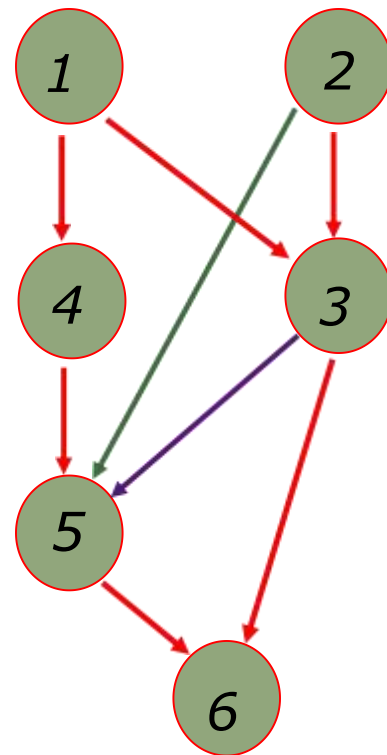
□ WAR hazards 不可能出现

- 因为是按顺序发送 + 发送时才读操作数



顺序发送的局限性

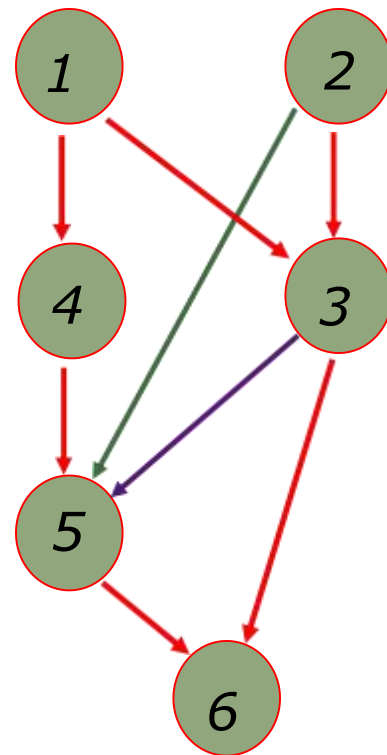
1	LD	F2,	34(R2)	latency 1	
2	LD	F4,	45(R3)	long	
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4,	F2,	F8	4
6	ADDD	F10,	F6,	F4	1



In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

顺序发送的局限性

1	LD	F2,	34(R2)	latency 1	
2	LD	F4,	45(R3)	long	
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4,	F2,	F8	4
6	ADDD	F10,	F6,	F4	1



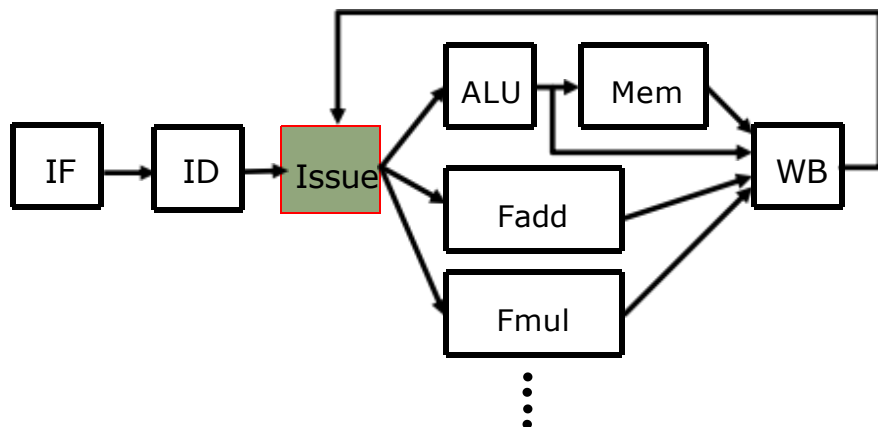
In-order:

1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

按顺序发射导致指令4在第3周期不能发射，实际上是可以发射的（操作数都准备好了）

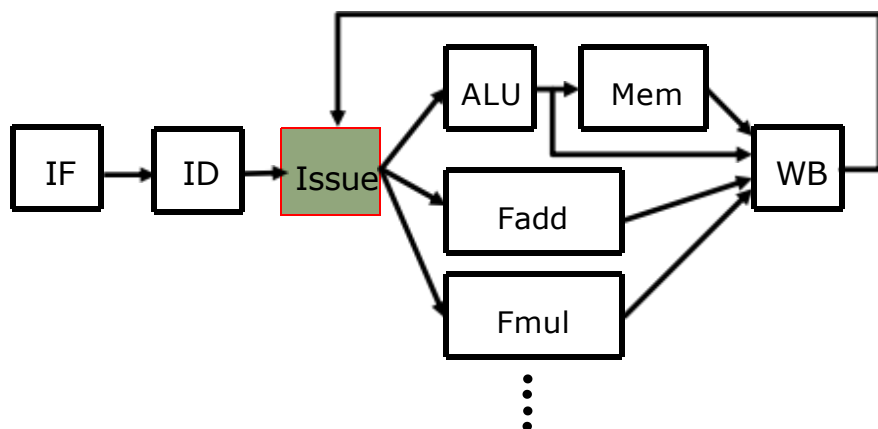
乱序发射

如何解决存在RAW相关性的指令停顿后，影响后面指令发射的问题？



做点其他事!

如何解决存在RAW相关性的指令停顿后，影响后面指令发射的问题？

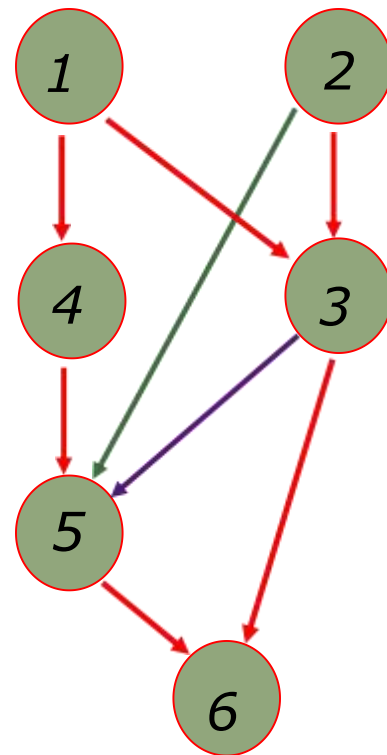


做点其他事!

- 发射阶段设置缓存，保存多条准备发射的指令
- Decode阶段将下一条指令加入缓存，如果：(1)缓存有空位；(2)指令不会引起WAR 或 WAW hazard.
- 对于缓存中的任何指令，只要其操作数准备好了就可以发射 (目前每个周期只能发射一条指令)

顺序发射 VS. 乱序发射

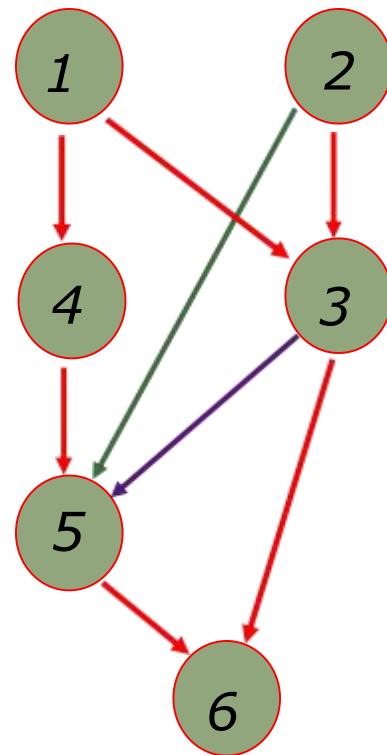
1	LD	F2,	34(R2)	<i>latency</i> 1	
2	LD	F4,	45(R3)	<i>long</i>	
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4,	F2,	F8	4
6	ADDD	F10,	F6,	F4	1



In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

顺序发射 VS. 乱序发射

1	LD	F2,	34(R2)	<i>latency</i> 1	
2	LD	F4,	45(R3)	<i>long</i>	
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4,	F2,	F8	4
6	ADDD	F10,	F6,	F4	1

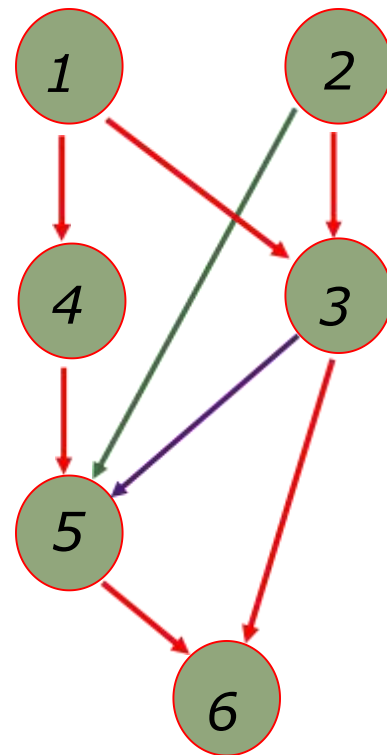


In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

Out-of-order: 1 (2,1) 4 4 2 3 5 . 3 . 5 6 6

顺序发射 VS. 乱序发射

1	LD	F2,	34(R2)	<i>latency</i> 1	
2	LD	F4,	45(R3)	<i>long</i>	
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4,	F2,	F8	4
6	ADDD	F10,	F6,	F4	1



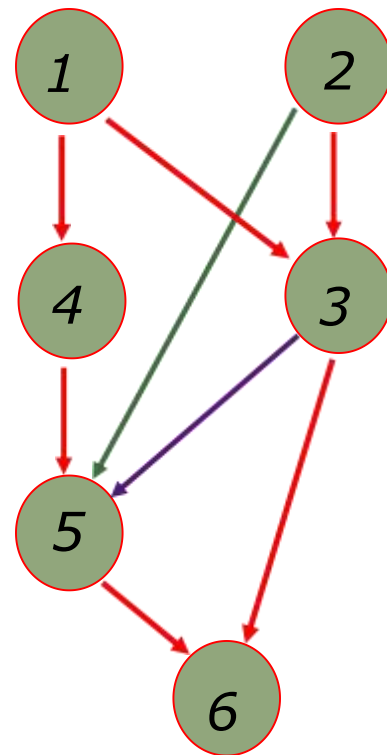
In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

Out-of-order: 1 (2,1) 4 4 2 3 5 . 3 . 5 6 6

乱序发射对性能并没有提高多少！

顺序发射 VS. 乱序发射

1	LD	F2,	34(R2)	<i>latency</i> 1	
2	LD	F4,	45(R3)	<i>long</i>	
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4,	F2,	F8	4
6	ADDD	F10,	F6,	F4	1



In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

Out-of-order: 1 (2,1) 4 4 2 3 5 . 3 . 5 6 6

WAR/WAW hazards 阻止了指令5发射

乱序发射对性能并没有提高多少！

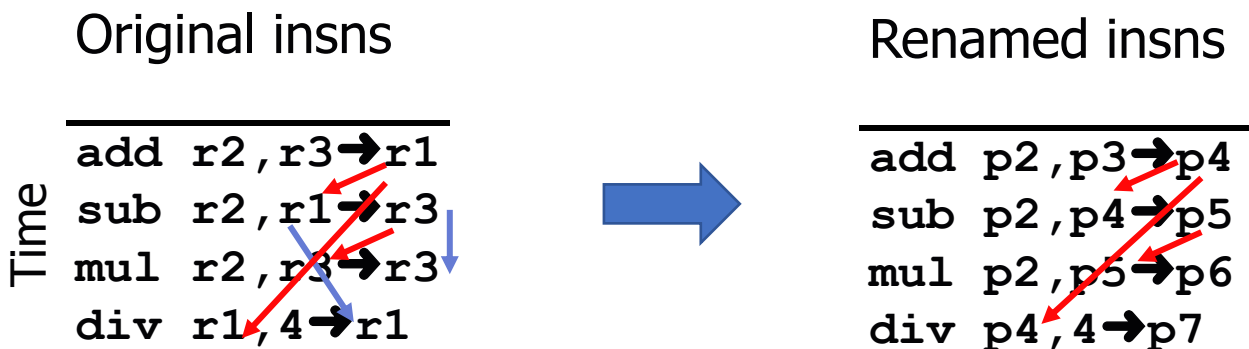
寄存器重命名 -> 消除WAR/WAW

□ WAR和WAW不是真正的相关

- 引起相关性的两个数其实并没有什么关联
- 之所以存在是因为寄存器的个数不够

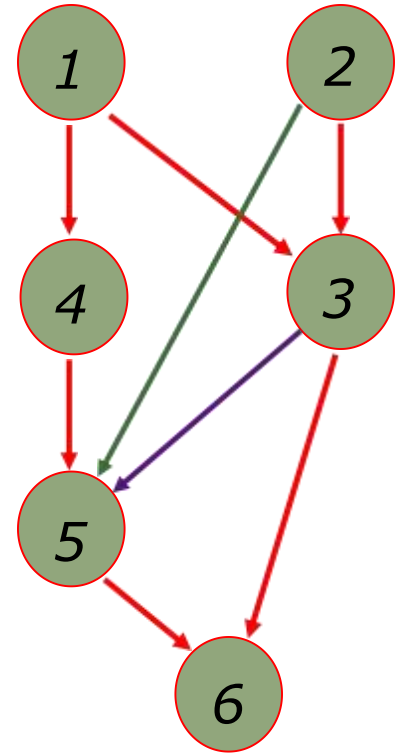
□ 如何通过寄存器重命名消除WAR和WAW

- 除了ISA提供的寄存器之外，需要大量的物理寄存器



寄存器重命名-例子

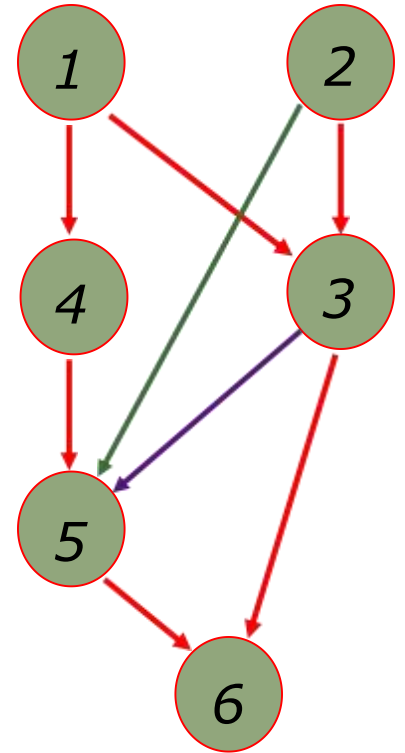
					<i>latency</i>
1	LD	F2,	34(R2)		1
2	LD	F4,	45(R3)		<i>long</i>
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4,	F2,	F8	4
6	ADDD	F10,	F6,	F4	1



In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

寄存器重命名-例子

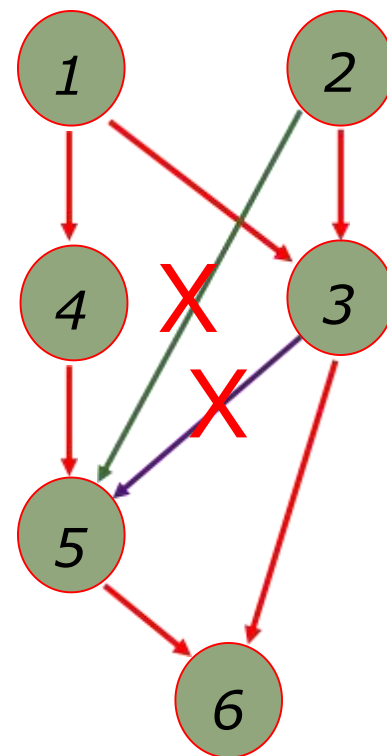
					<i>latency</i>
1	LD	F2,	34(R2)		1
2	LD	F4,	45(R3)		<i>long</i>
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4' ,	F2,	F8	4
6	ADDD	F10,	F6,	F4'	1



In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

寄存器重命名-例子

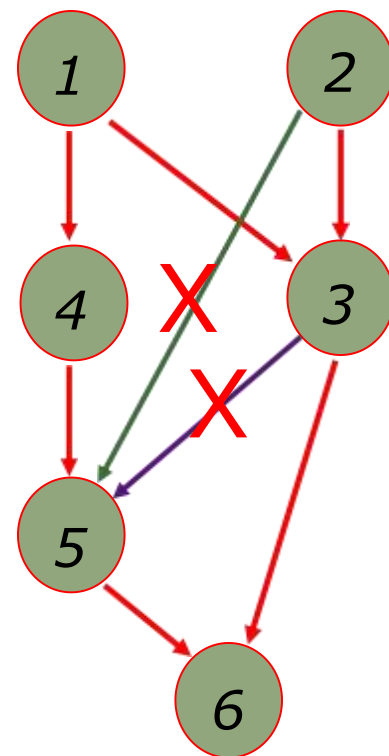
1	LD	F2,	34(R2)	latency 1	
2	LD	F4,	45(R3)	long	
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4',	F2,	F8	4
6	ADDD	F10,	F6,	F4'	1



In-order: 1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

寄存器重命名-例子

					<i>latency</i>
1	LD	F2,	34(R2)		1
2	LD	F4,	45(R3)		<i>long</i>
3	MULTD	F6,	F4,	F2	3
4	SUBD	F8,	F2,	F2	1
5	DIVD	F4' ,	F2,	F8	4
6	ADDD	F10,	F6,	F4'	1



In-order:

1 (2,1) 2 3 4 4 3 5 . . . 5 6 6

Out-of-order:

1 (2,1) 4 4 5 . . . (2,5) 3 . . 3 6 6

乱序执行 (Tomasulo算法)

乱序执行 (Out of Order Execution)

□ 也称为指令动态调度 (vs. 指令静态调度)

- 由硬件在运行时完成

□ 简单描述:

1. 指令译码完成后，放入保留站（指令缓存空间）
2. 每个周期，检测保留站中每条指令的源操作数
3. 如果指令的源操作数都准备好了，发送指令到执行单元（如果有空闲）
4. 指令完成后，从保留站删除

□ 乱序执行的好处：

- 当某条指令的执行时间很长时，允许后面的不想关指令先执行
- 有相关性的指令给无相关性的指令让路

乱序执行的条件

1. 需要为数据的生产方和消费方建立关联关系

- 寄存器重命名：每个数据都和一个“tag”关联

2. 需要缓存那些操作数还没有准备好的指令

- 放到保留站

3. 指令需要检测其操作数状态（是否准备好）

- 当数据准备好时，会广播其“tag”
- 所有指令将其tag与广播的tag进行比较，如果相同，表明数据准备好

4. 当所有操作数都准备好时，需要将指令发送到执行单元

- 当所有操作数准备好时，指令会被唤醒
- 如果多条指令同时唤醒，需要选择一条发送到执行单元

Tomasulo 算法

□ 经典的乱序执行硬件实现方法

- 在IBM 360/91 Floating Point Units中使用
- 参考文献: Tomasulo, “**An Efficient Algorithm for Exploiting Multiple Arithmetic Units,**” IBM Journal of R&D, Jan. 1967.

□ Tomasulo算法的变形应用在很多高性能处理器中

- Intel Pentium Pro, AMD K5
- Alpha 21264, MIPS R10000, IBM POWER5, IBM z196, Oracle UltraSPARC T4, ARM Cortex A15

Tomasulo 算法

核心思想：

- 每个计算单元维护一个保留站(一组物理寄存器)
 - 用于缓存操作数还没有准备好的指令
- 通过寄存器重命名消除WAW and WAR相关性
 - 将寄存器ID重命名为保留站ID
- 乱序发送指令

Tomasulo 算法

如果保留站仍有空余表项

- 将指令送入保留站，并进行寄存器重命名

否则 stall

对于在保留站中的每一条指令:

- 监听总线
- 当发现操作数的tag时, 获取数据并保存在保留站中
- 当所有操作数都准备好了, 指令变为可发送

发送指令到运算单元

当指令运算结束

- 将计算结果在总线上广播
- 寄存器文件连接着总线
 - 寄存器增加了一个tag，记录该寄存器在等待哪个保留站ID的值
 - 如果寄存器的tag和总线的tag匹配上了，将值写入寄存器 (同时清除tag)
- 回收保留站表项（清除该指令）

Tomasulo 算法示例

Reservation Station (保留站)

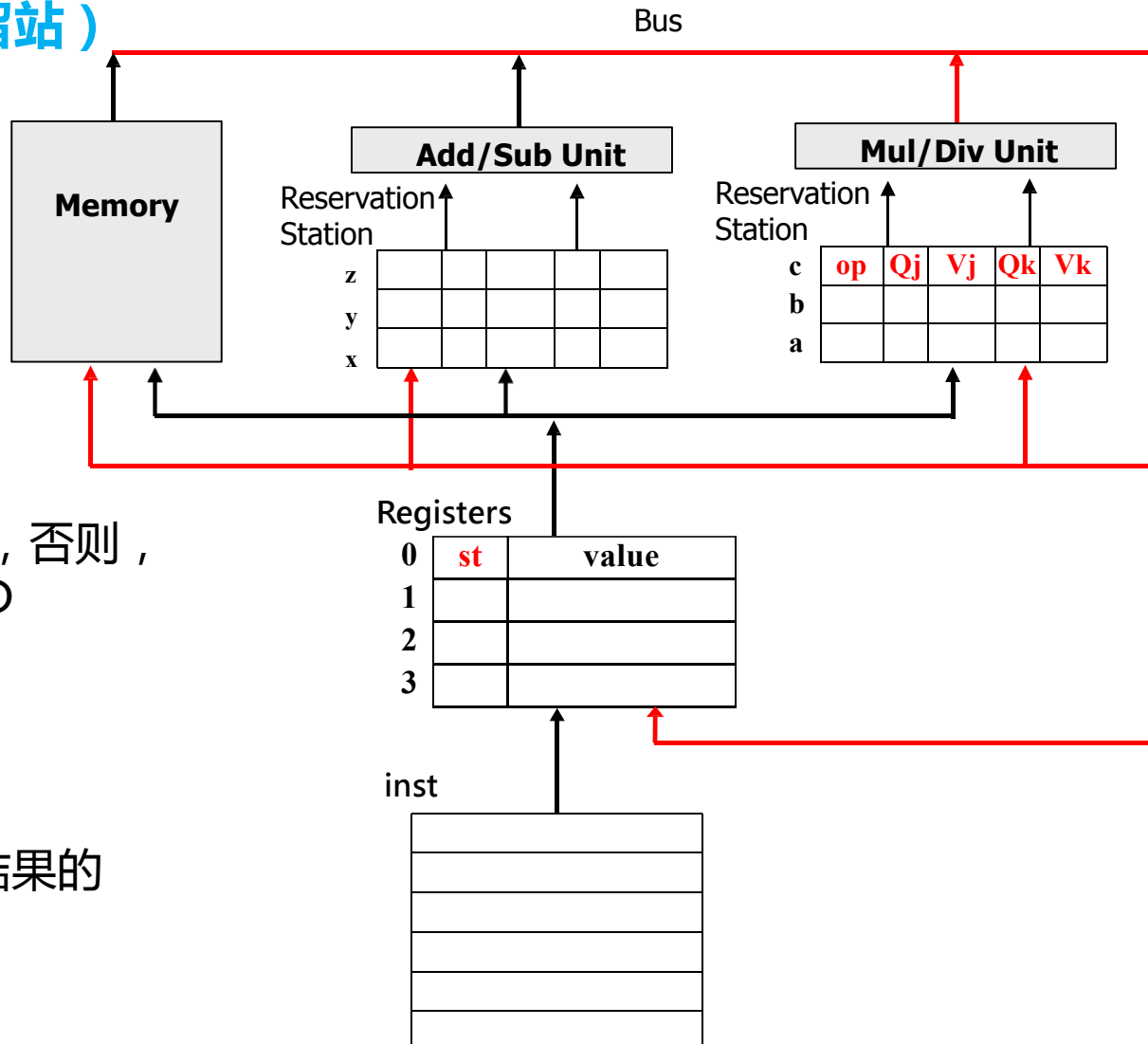
- **Op**: 指令的op code
- **Vj, Vk**: 操作数的值
- **Qj, Qk**: 将要提供操作数值的保留站ID(0表示准备好了)

Registers (增加一列):

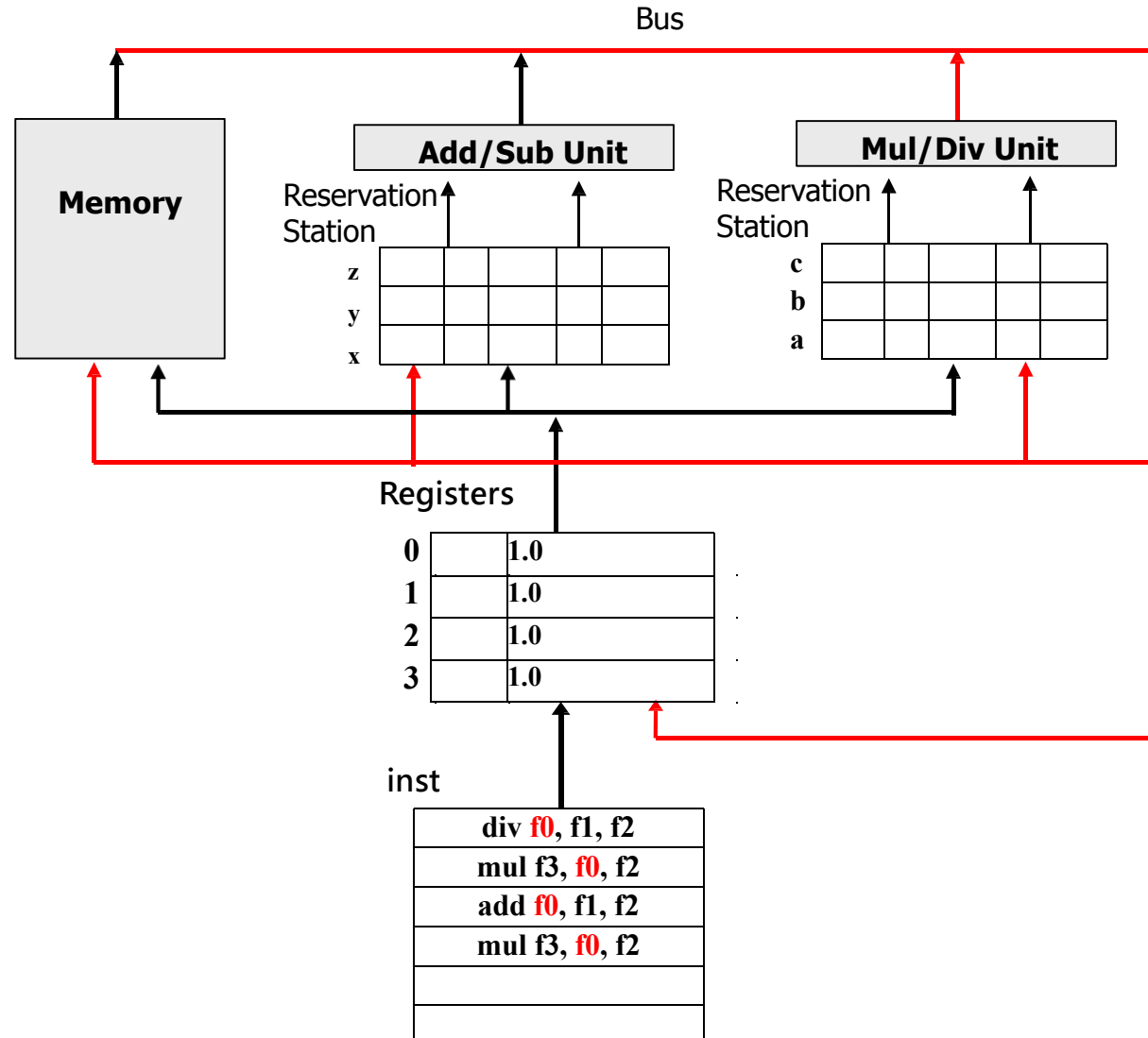
- **st**: “空” 表示该值可以使用，否则，记录将要提供该值的保留站ID

Bus

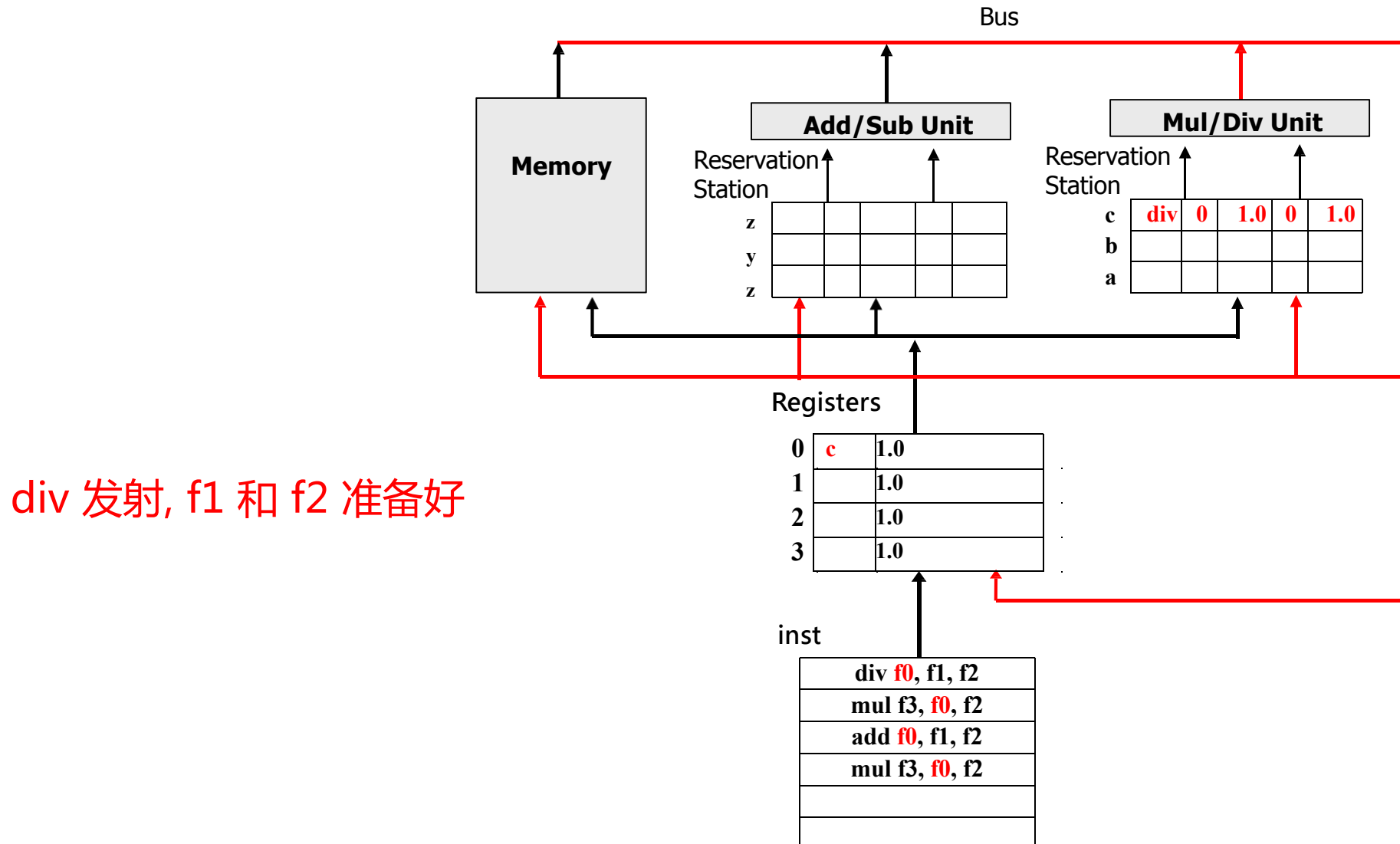
- 广播结果，同时带上提供该结果的保留站ID



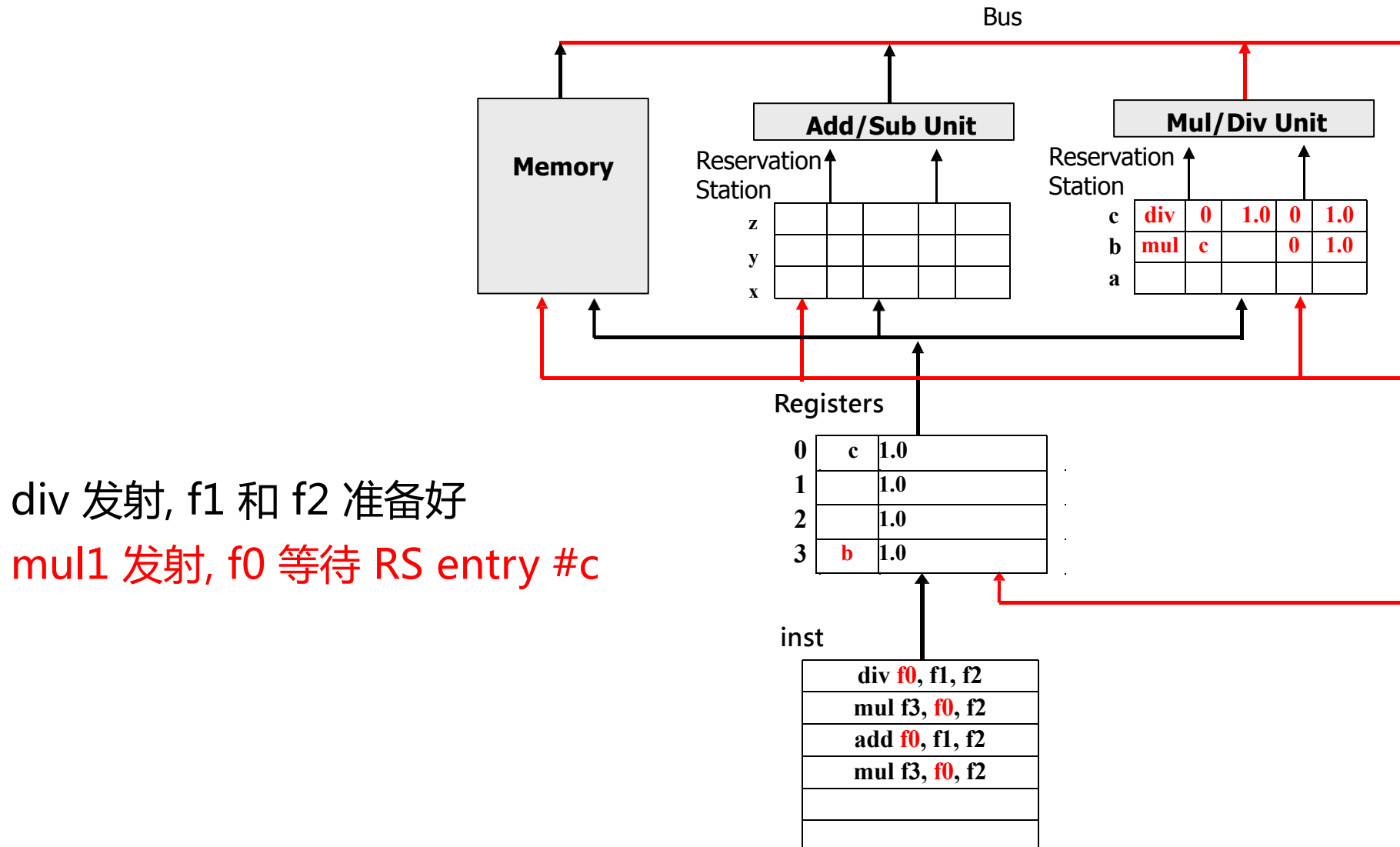
Tomasulo 算法示例



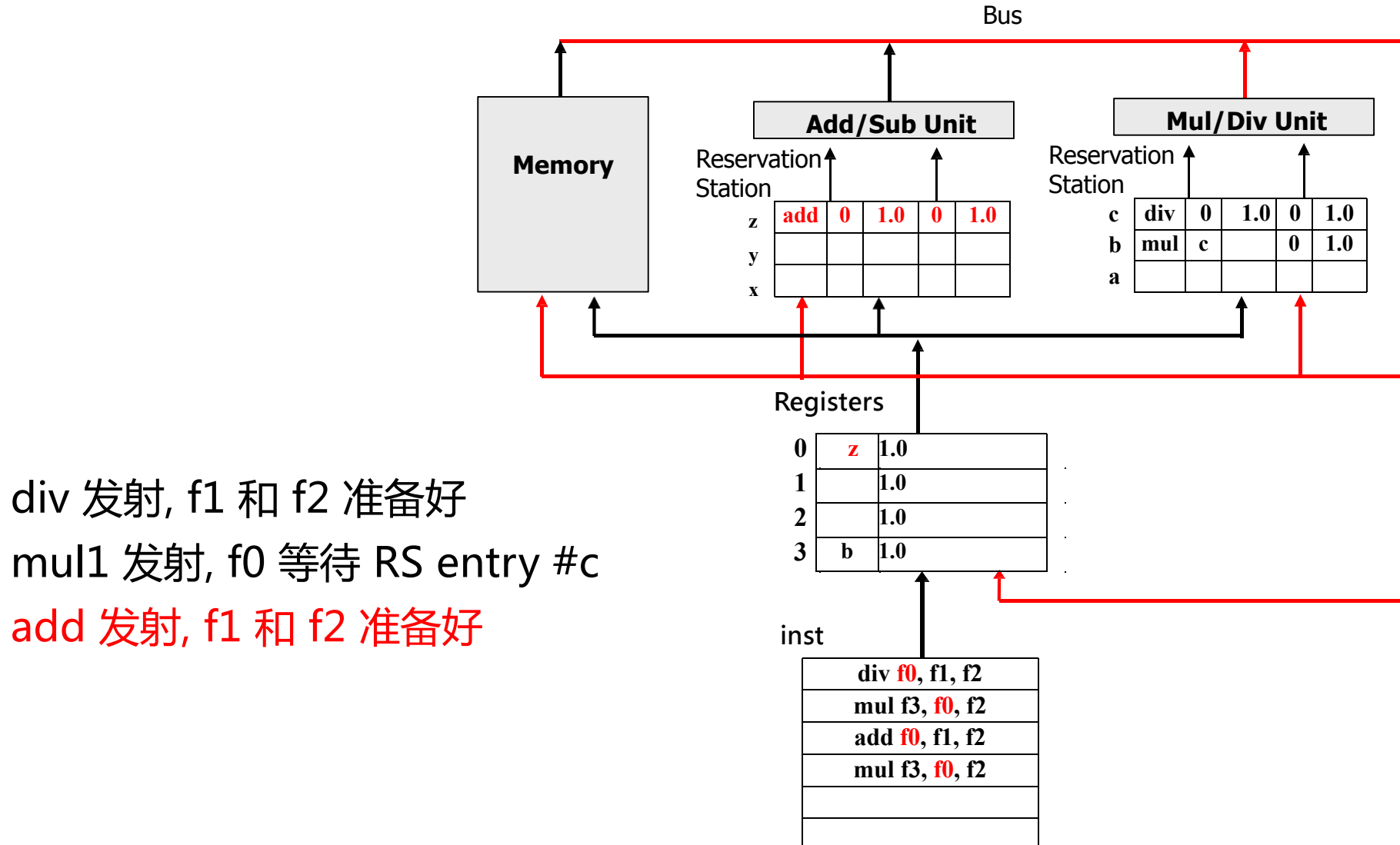
Tomasulo 算法示例



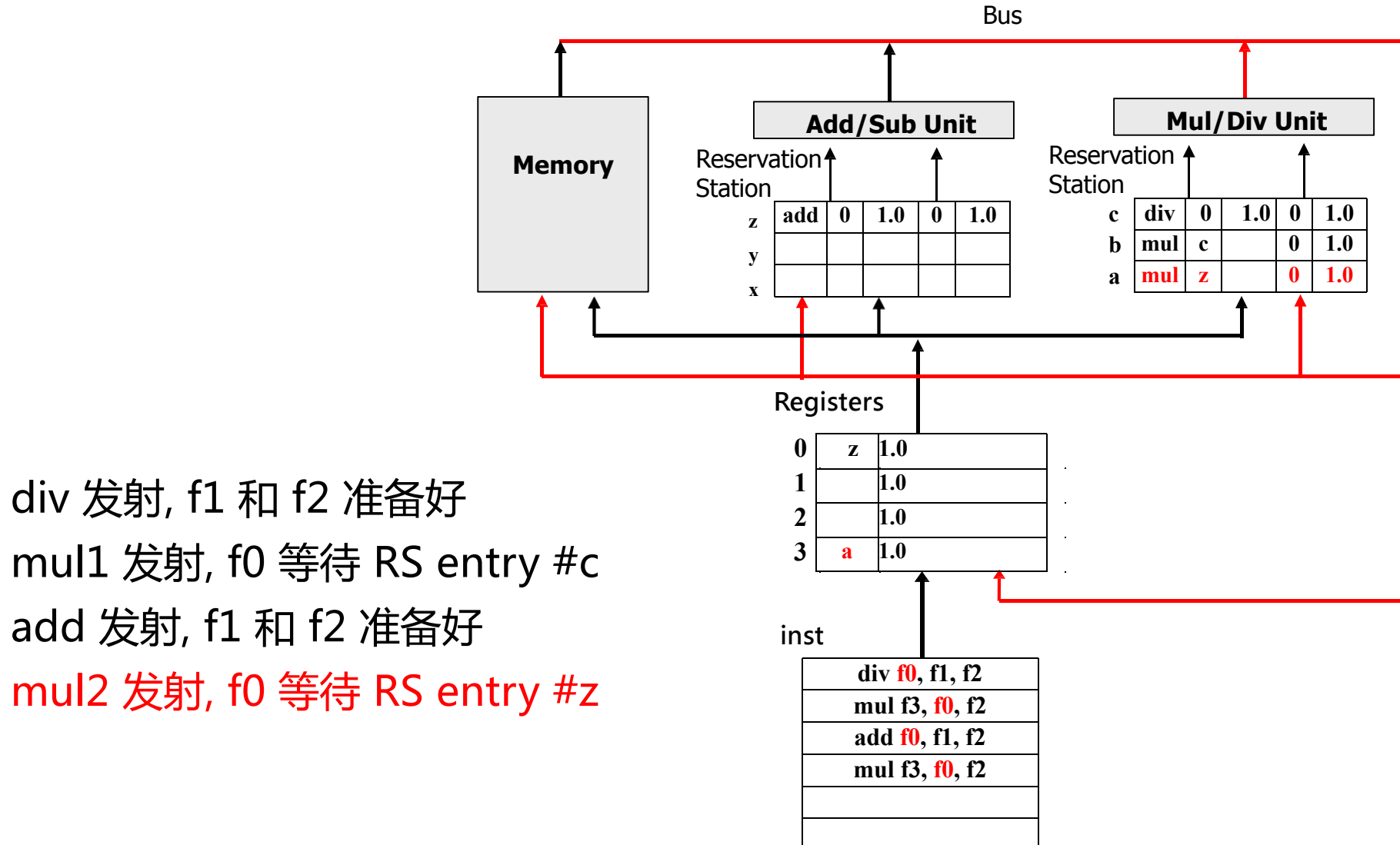
Tomasulo 算法示例



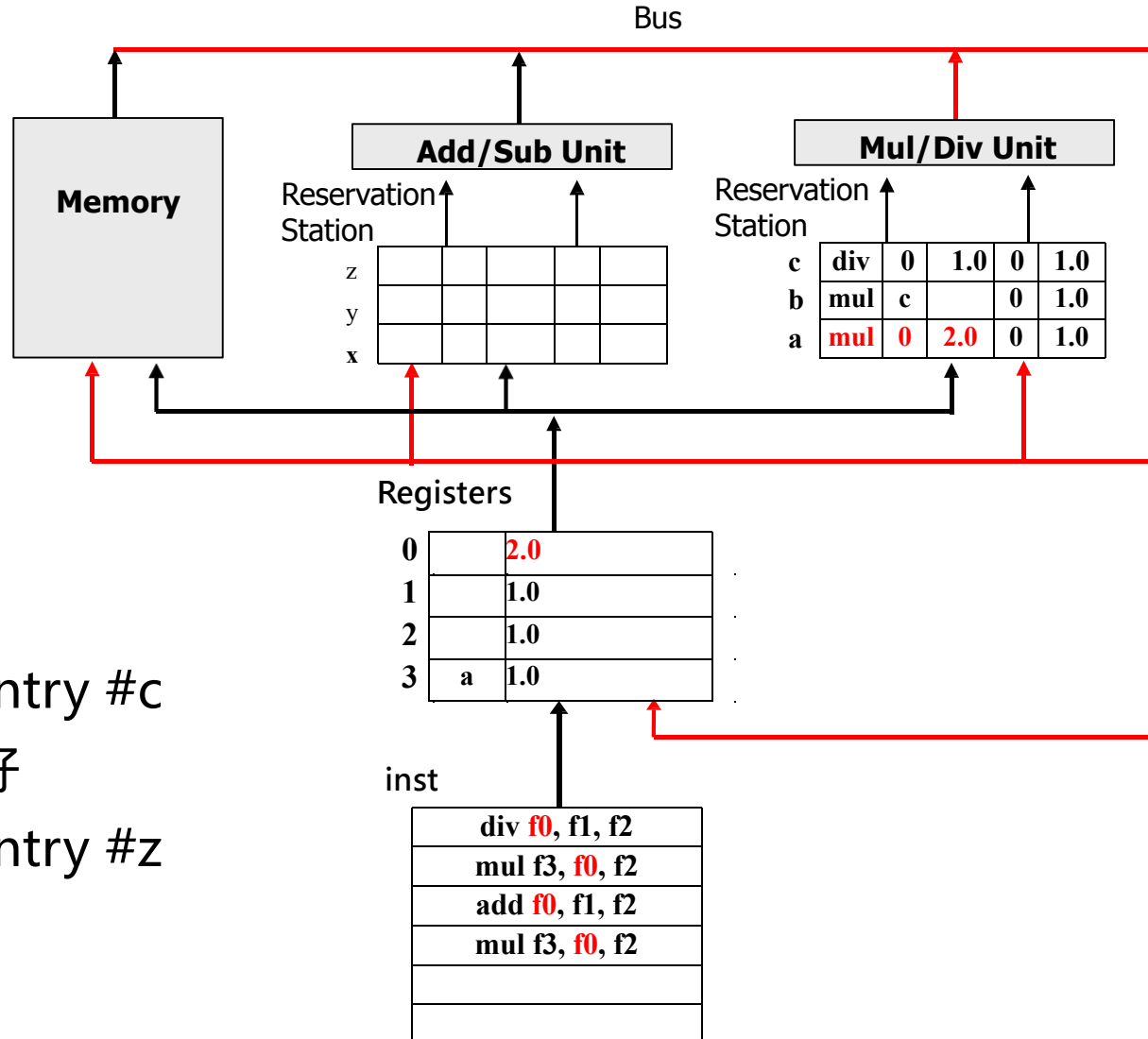
Tomasulo 算法示例



Tomasulo 算法示例

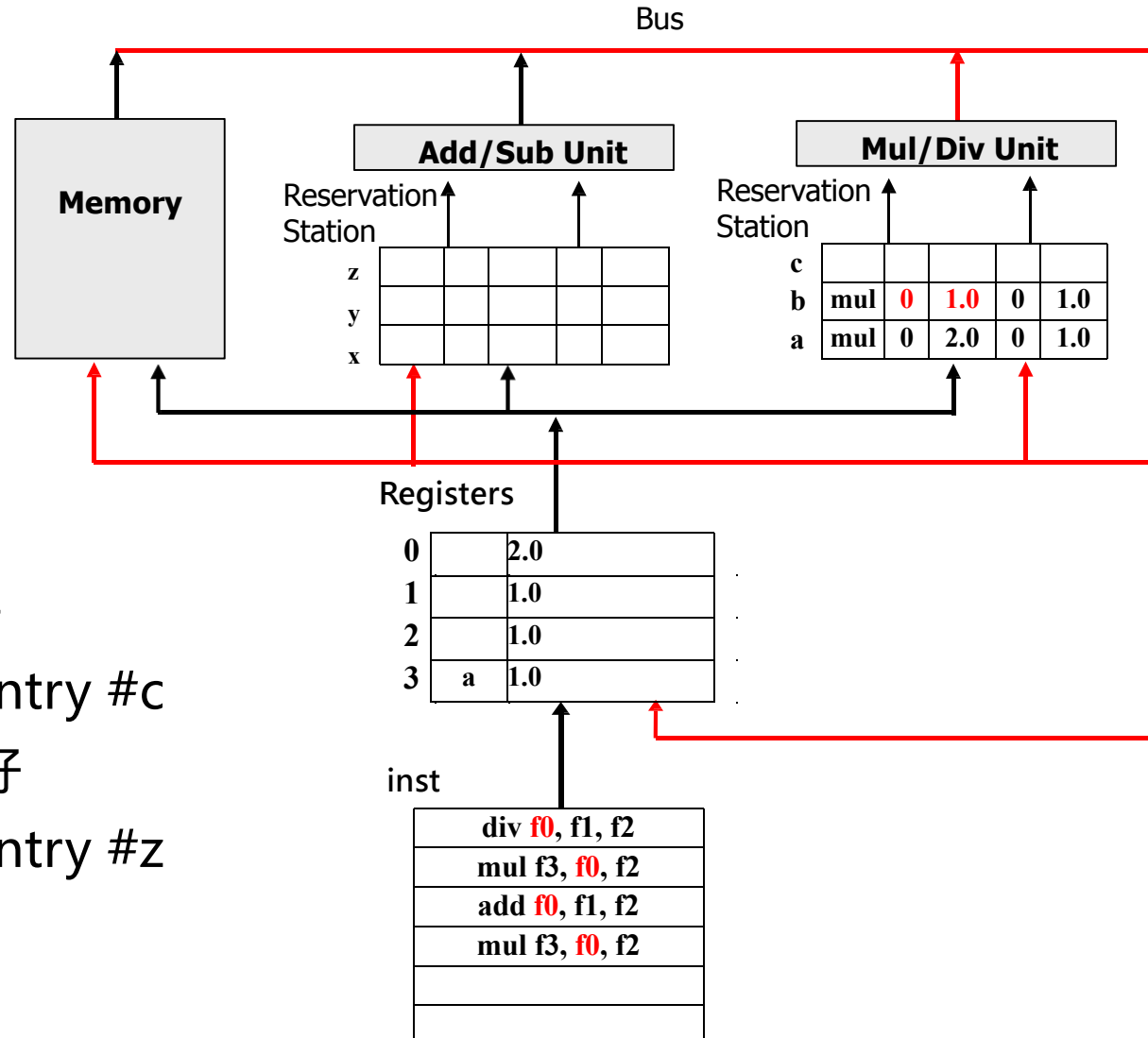


Tomasulo 算法示例



div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 RS entry #c
add 发射, f1 和 f2 准备好
mul2 发射, f0 等待 RS entry #z
add 写回

Tomasulo 算法示例



div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 RS entry #c
add 发射, f1 和 f2 准备好
mul2 发射, f0 等待 RS entry #z
add 写回
div 写回

精确异常

(Precise Exception)

流水线中的异常

- ✓ I/O 请求：外部中断
- ✓ 指令异常：用户请求中断
 - sys call、breakpoint、trace and debug
- ✓ 运算单元
 - overflow、float exception
- ✓ 存储单元
 - unaligned memory address、invalid access to system space、TLB miss、page fault
- ✓ 指令错误
 - unknown inst
- ✓ 硬件错误
- ✓ etc

精确异常(precise exception)

- **精确异常：当异常发生时**, 异常之前的指令应该执行完，异常之后的指令不应该执行
- **非精确异常: 乱序执行**，后面的指令比前面的指令先执行完，修改了系统状态 (registers or memory)
 - **Tomasulo 算法不能保证精确异常**

乱序执行如何保证精确异常？

- **思路:** 利用缓存临时记录执行结果，等到前面所有指令提交之后，再提交该指令结果
- **Reorder Buffer (ROB):** 缓存执行结果
 - 当一条指令进行写回时，将结果放到ROB
 - 增加一个**Commit 阶段**，将ROB中的结果写到 register/memory
 - 一条指令只有在其前面所有指令的结果都commit了，才能进行commit

总结：乱序执行，但顺序提交 (Commit)

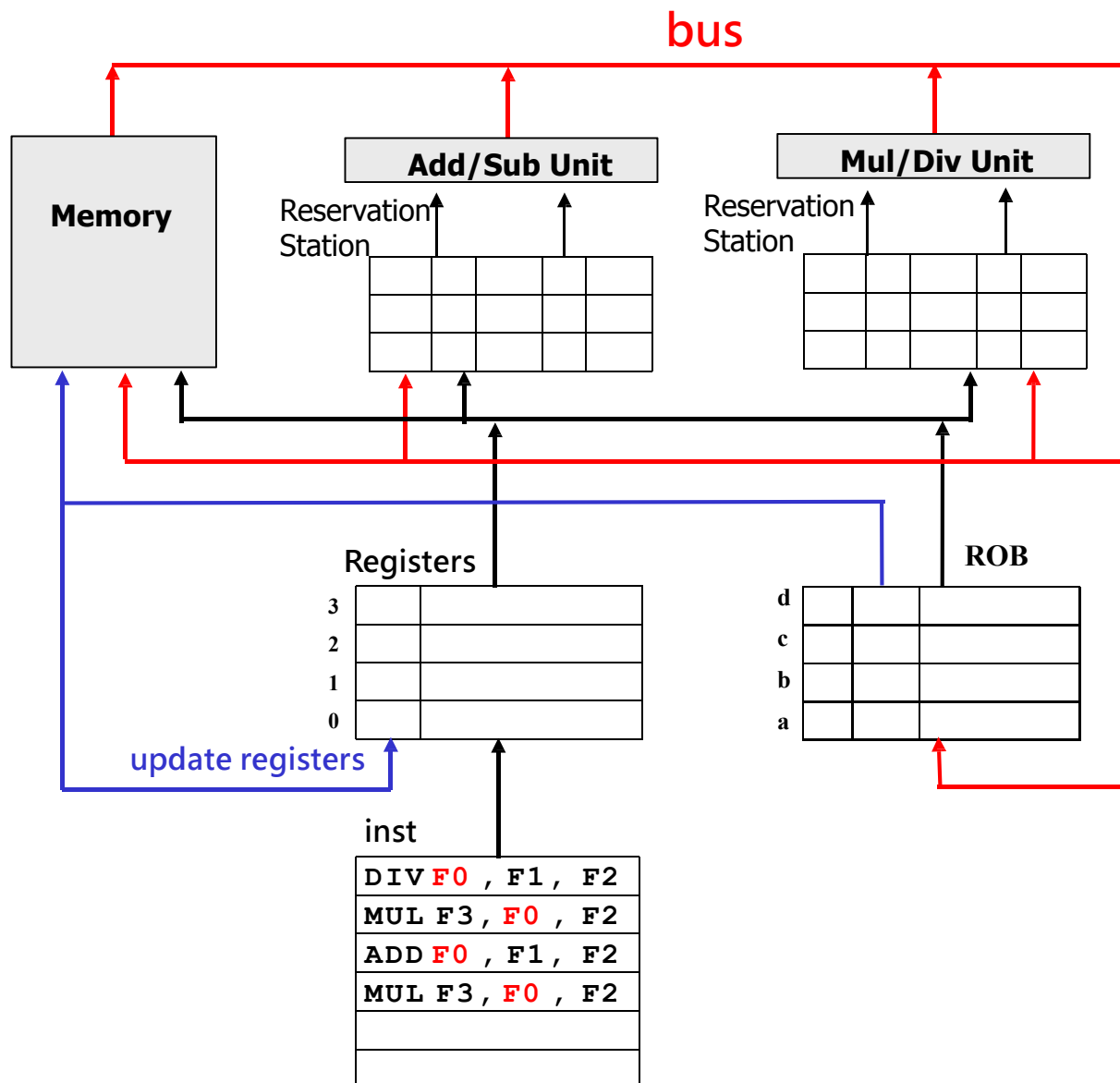
Reorder Buffer (ROB)

每一项内容: 目标寄存器ID、指令结果、指令Op

- 指令进入保留站的同时，也进入ROB
- **寄存器ID 重命名为ROB ID (不是保留站ID)**
- 指令在写回阶段将结果写入ROB, 而不是寄存器/内存
 - ✓ 后面的指令可以从ROB读取这些结果
- 当指令commit时，才将结果写到寄存器/内存
- 指令commit后，将指令从ROB删除

只要指令尚未commit, 就不会修改系统状态, 这样就很容易取消指令 (比如遇到异常)

带ROB的乱序执行



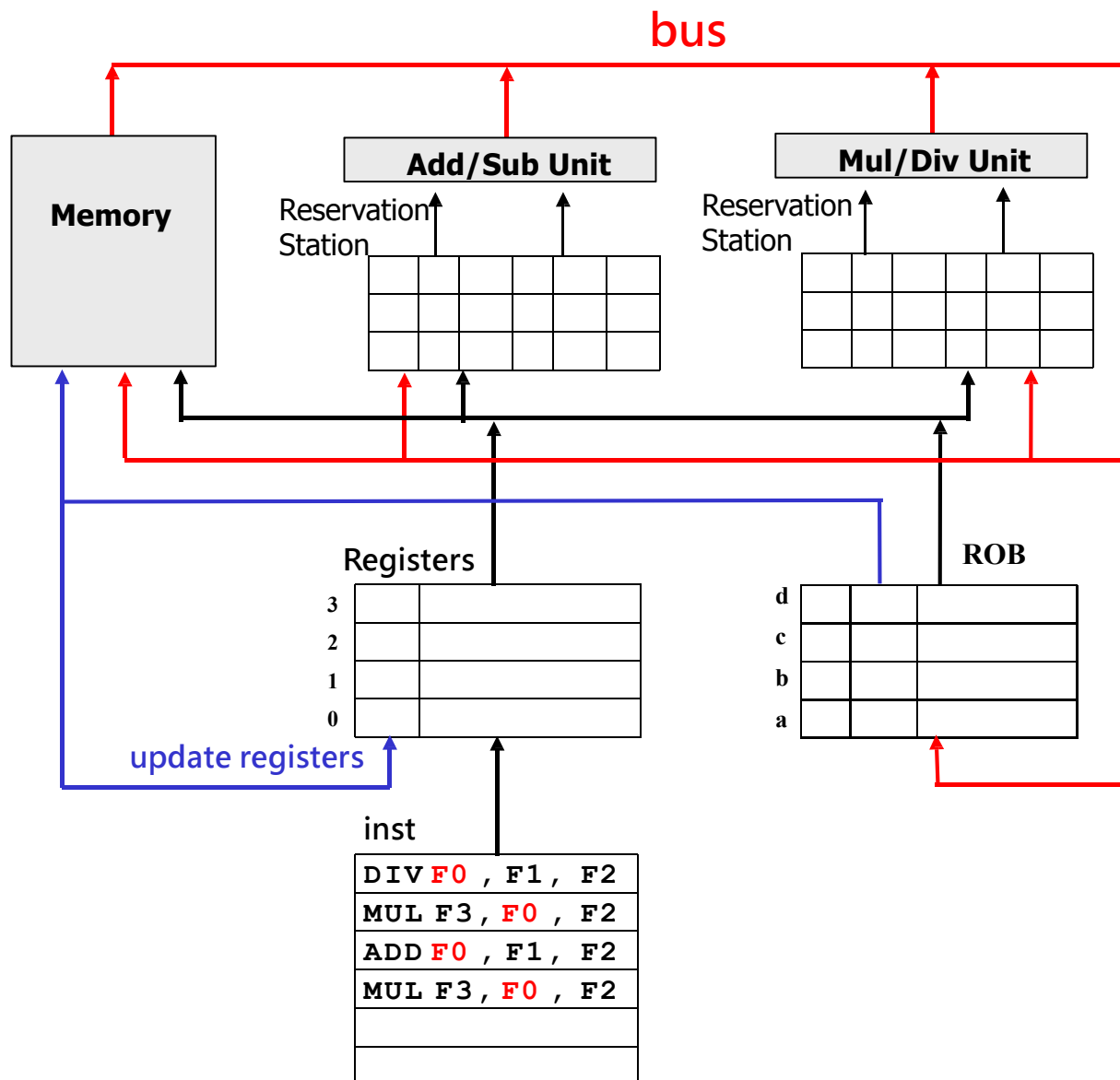
带ROB的乱序执行

DIV F0, F1, F2

MUL F3, F0, F2

ADD F0, F1, F2

MUL F3, F0, F2



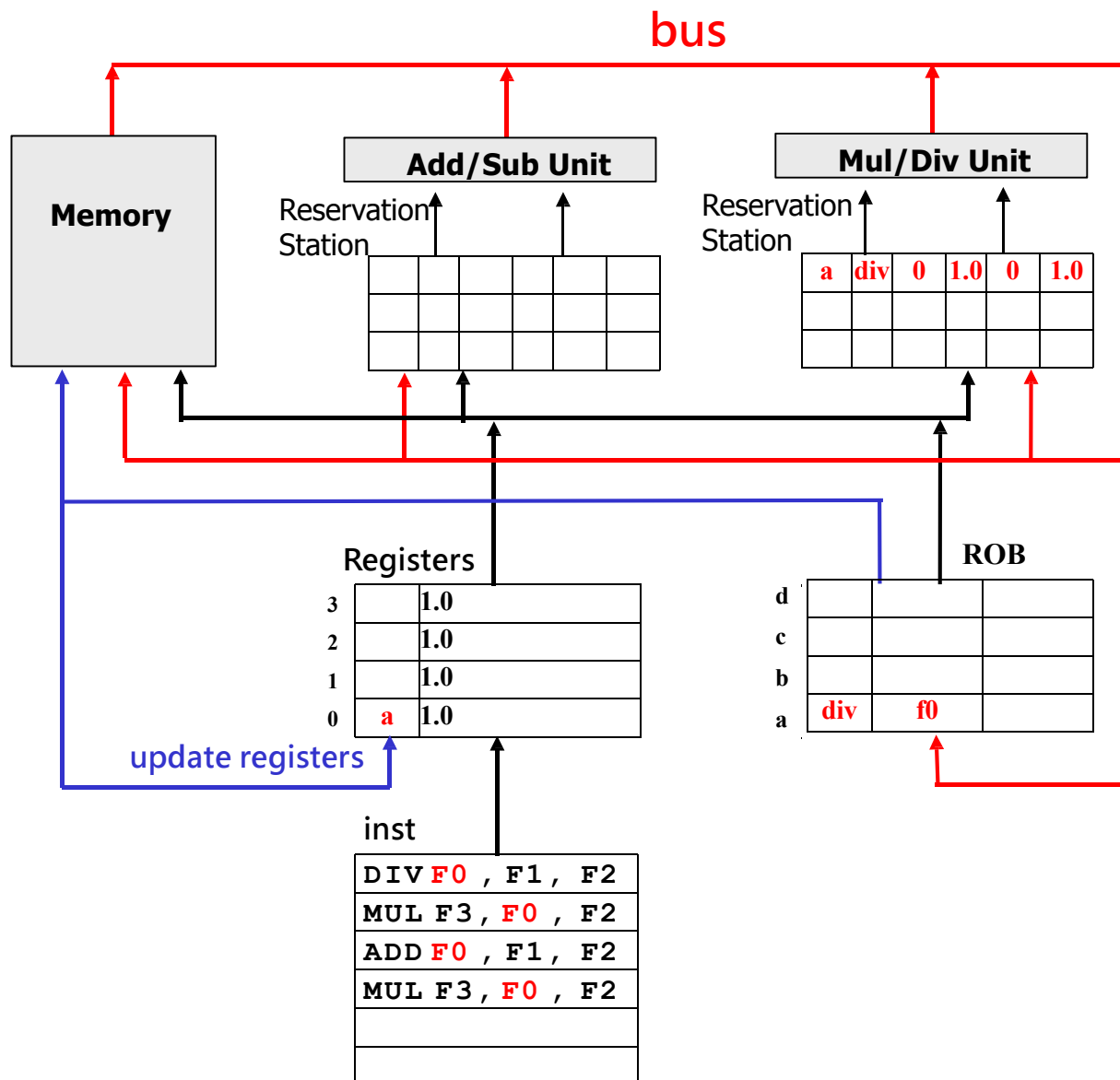
带ROB的乱序执行

DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2



div 发射, f1 和 f2 准备好

带ROB的乱序执行

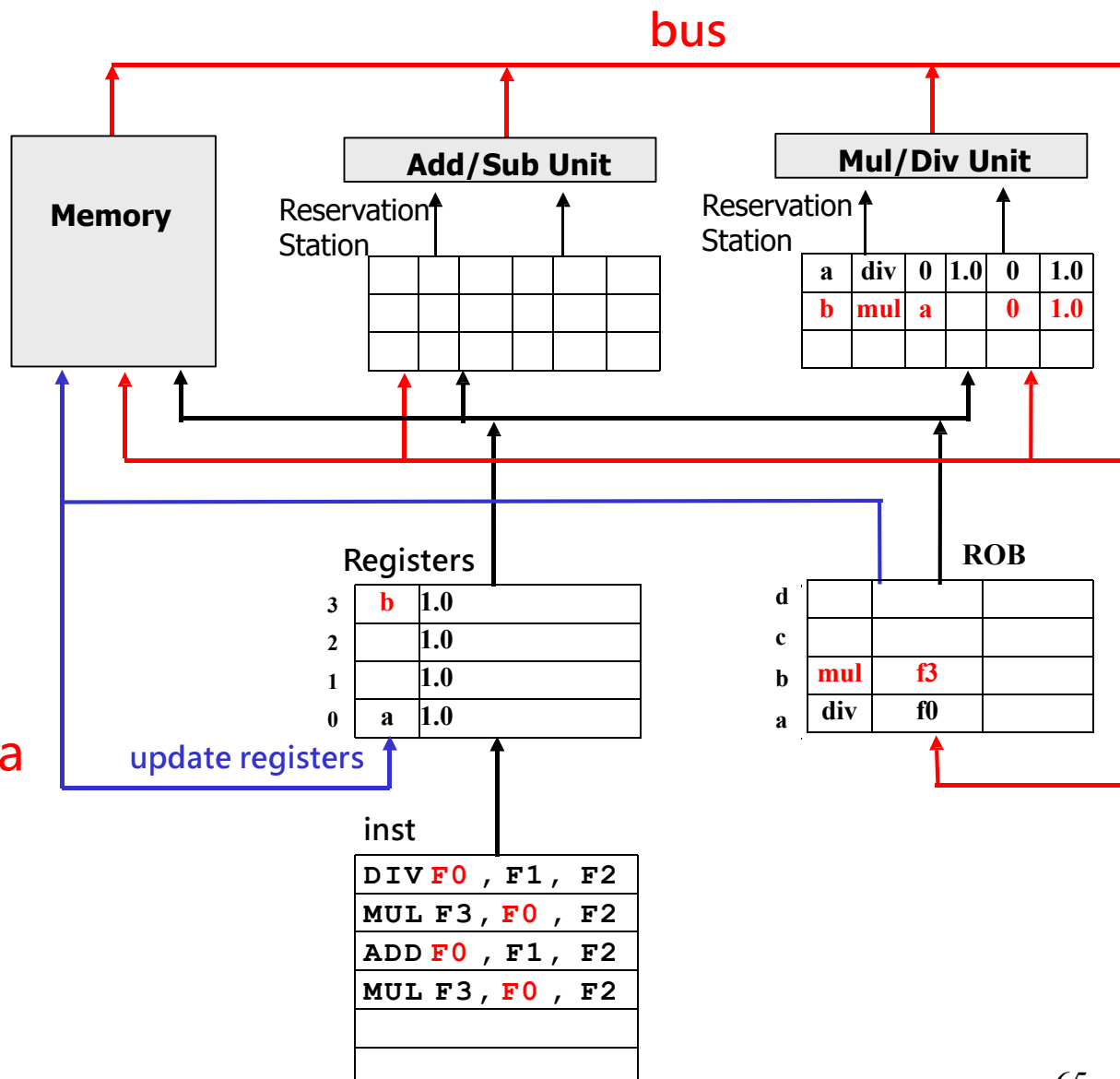
DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2

div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 ROB #a



带ROB的乱序执行

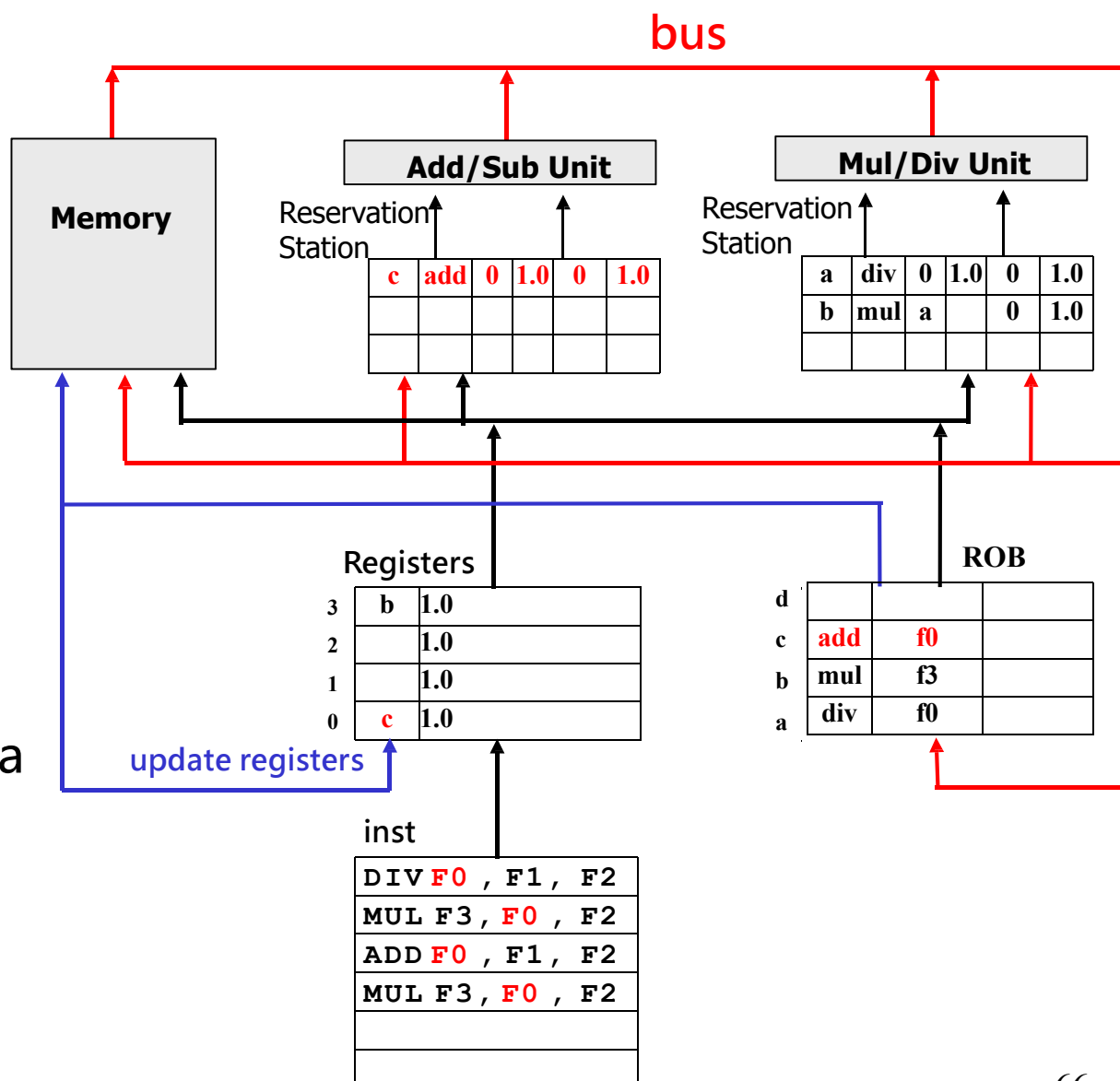
DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2

div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 ROB #a
add 发射, f1 和 f2 准备好



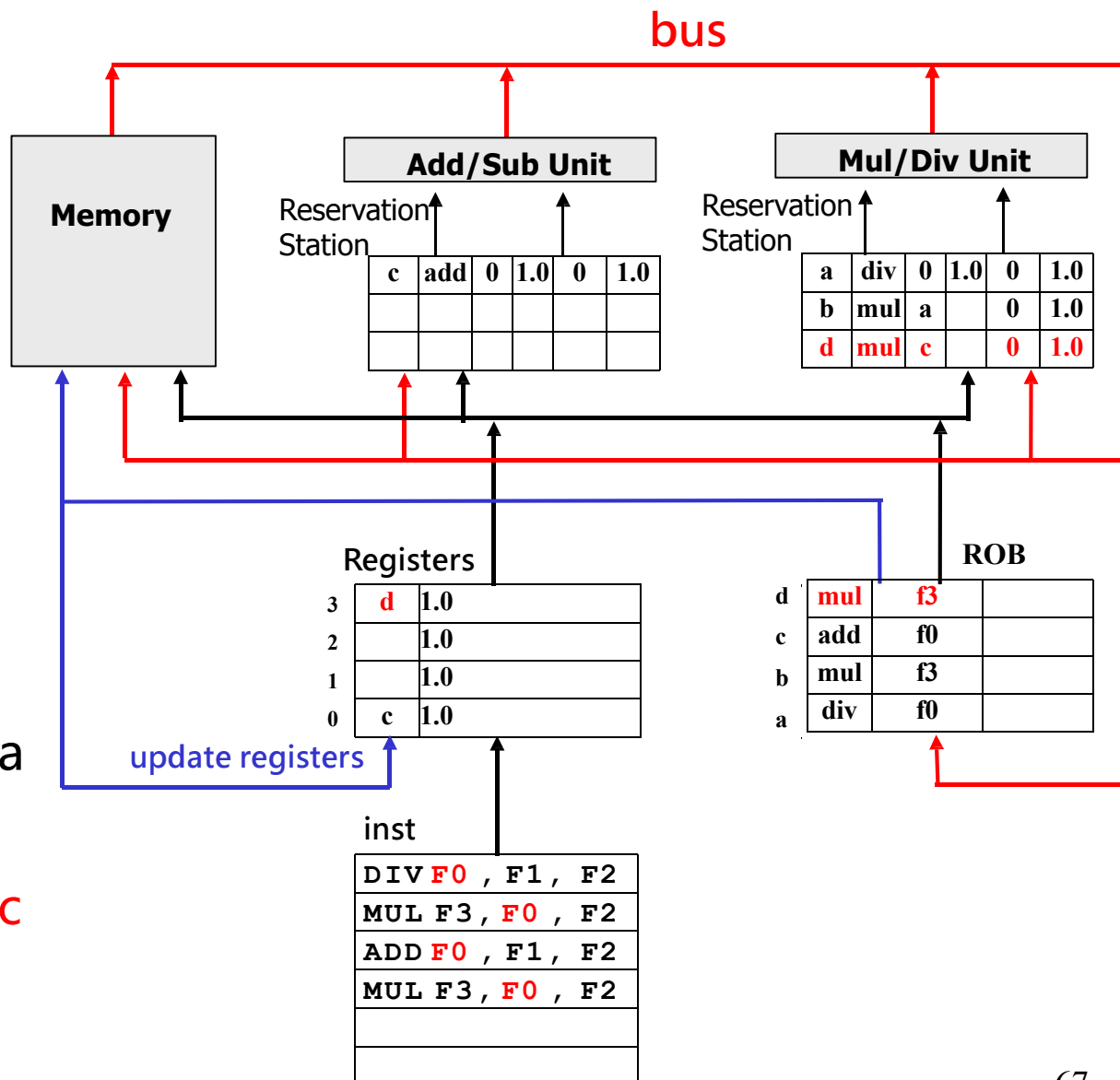
带ROB的乱序执行

DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2



div 发射, f1 和 f2 准备好
 mul1 发射, f0 等待 ROB #a
 add 发射, f1 和 f2 准备好
 mul2 发射, f0 等待 ROB #c

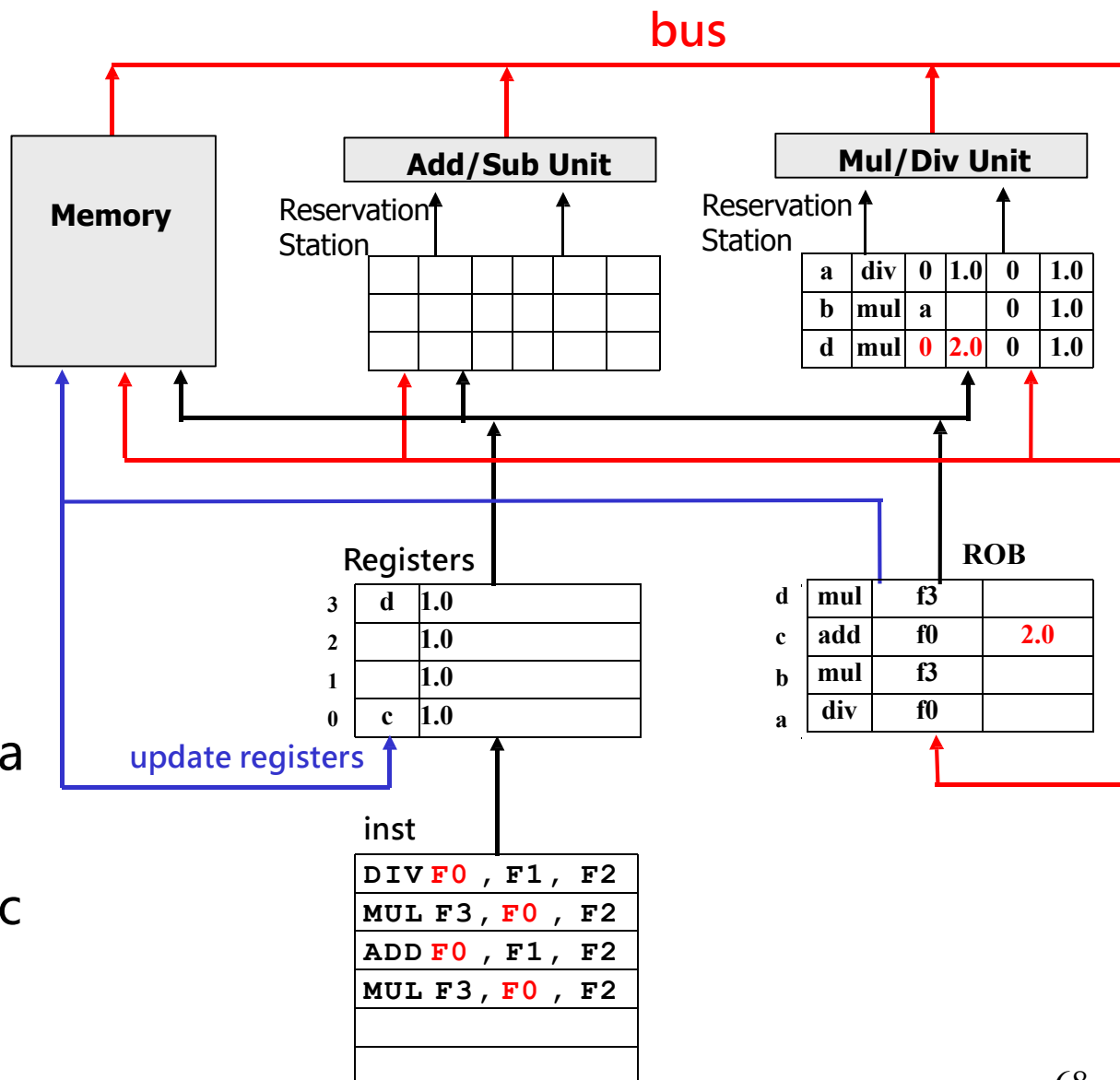
带ROB的乱序执行

DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2



div 发射, f1 和 f2 准备好
 mul1 发射, f0 等待 ROB #a
 add 发射, f1 和 f2 准备好
 mul2 发射, f0 等待 ROB #c
 add 写回

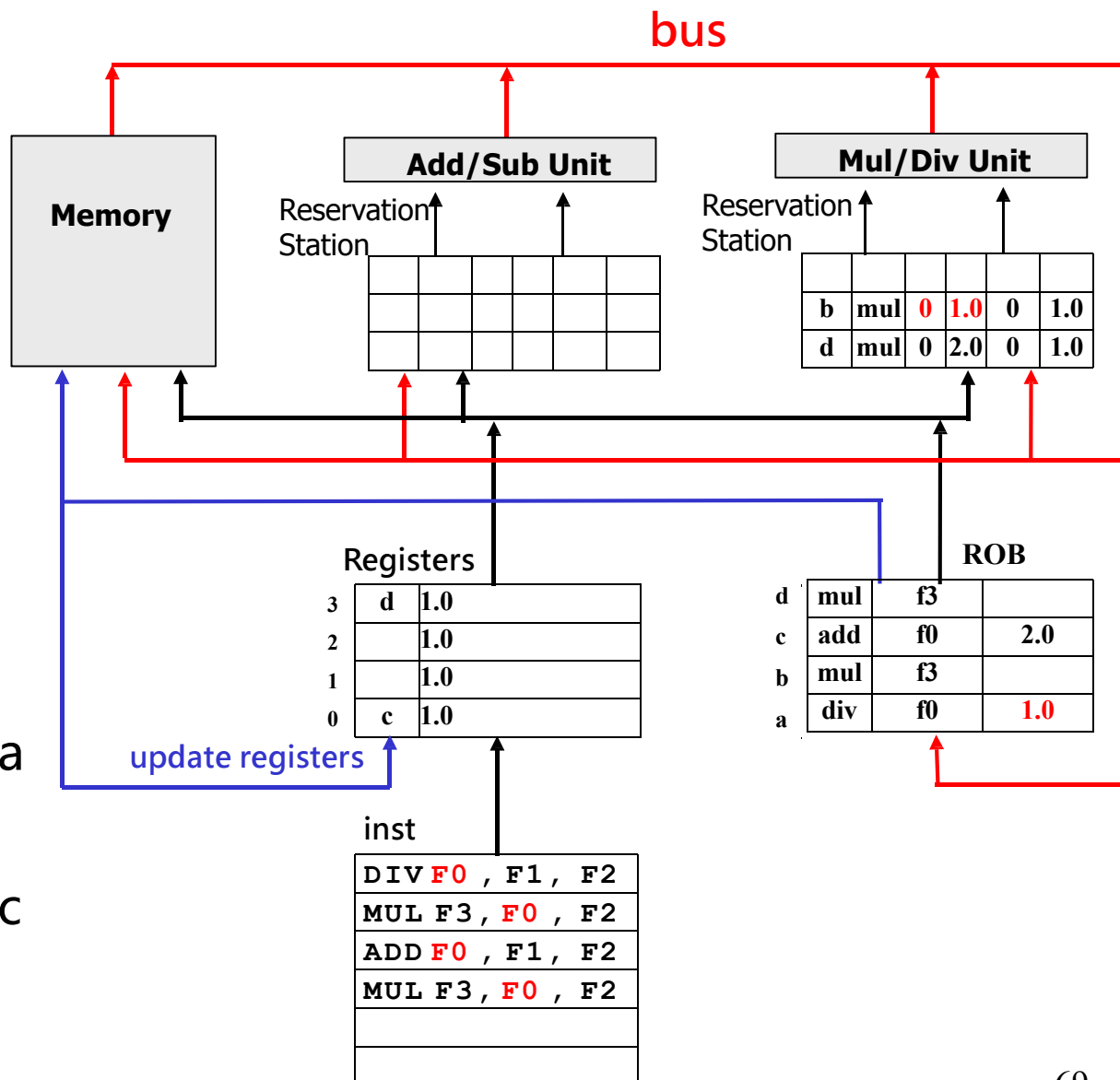
带ROB的乱序执行

DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2



div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 ROB #a
add 发射, f1 和 f2 准备好
mul2 发射, f0 等待 ROB #c
add 写回
div 写回

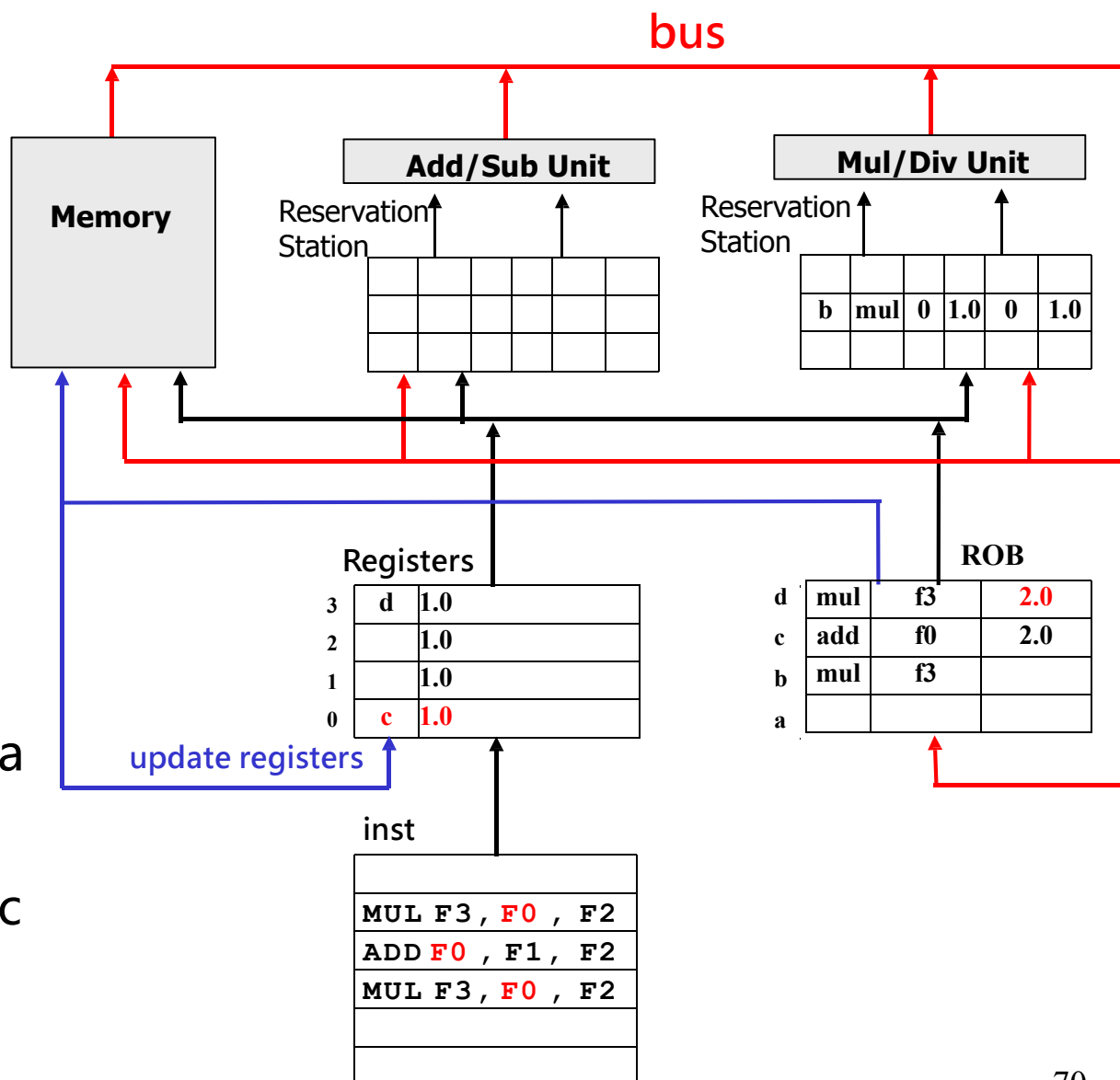
带ROB的乱序执行

DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2



div 发射, f1 和 f2 准备好
 mul1 发射, f0 等待 ROB #a
 add 发射, f1 和 f2 准备好
 mul2 发射, f0 等待 ROB #c
 add 写回
 div 写回

div 提交, mul2 写回

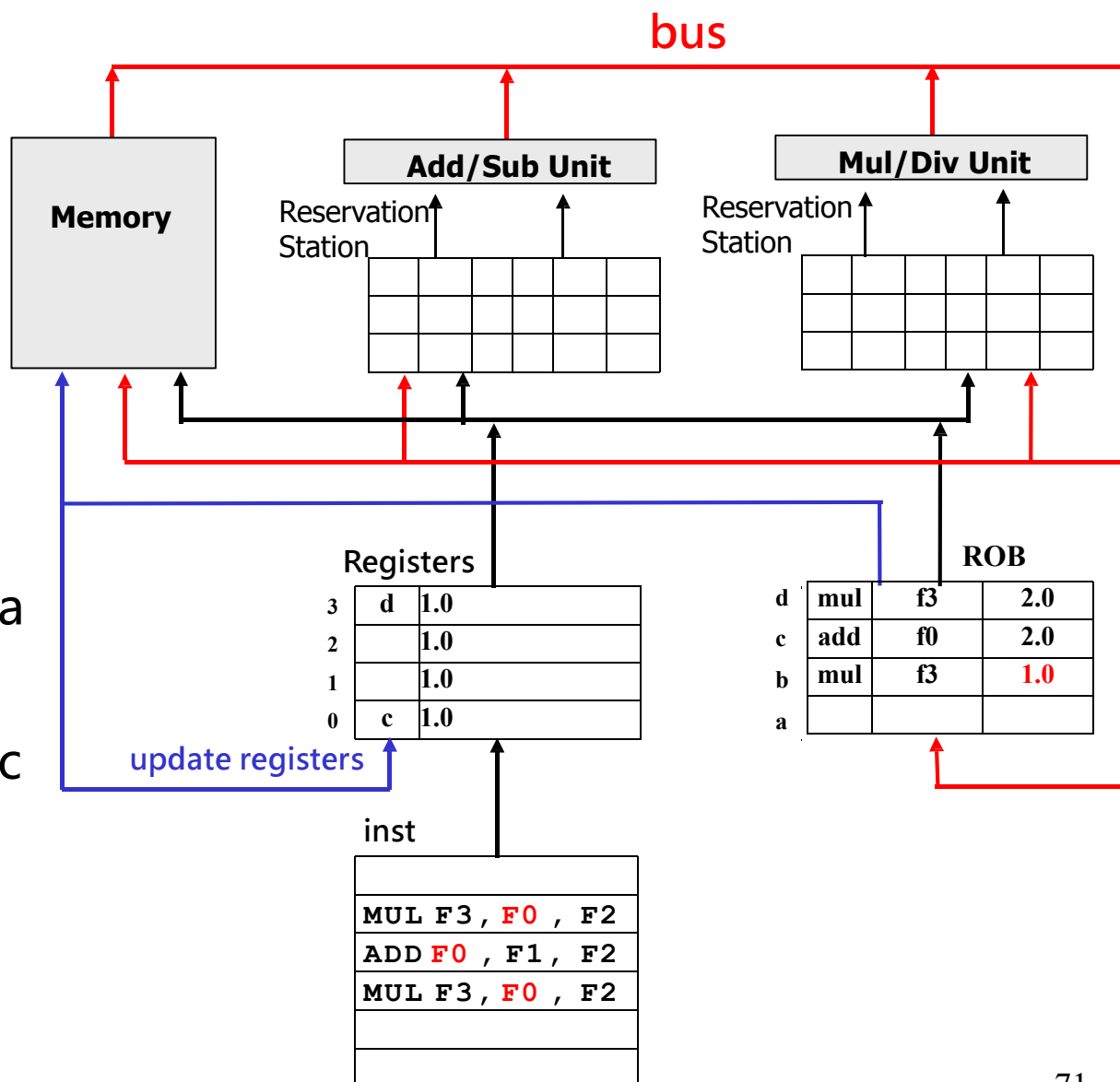
带ROB的乱序执行

DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2



div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 ROB #a
add 发射, f1 和 f2 准备好
mul2 发射, f0 等待 ROB #c
add 写回
div 写回
div 提交, mul2 写回
mul 写回

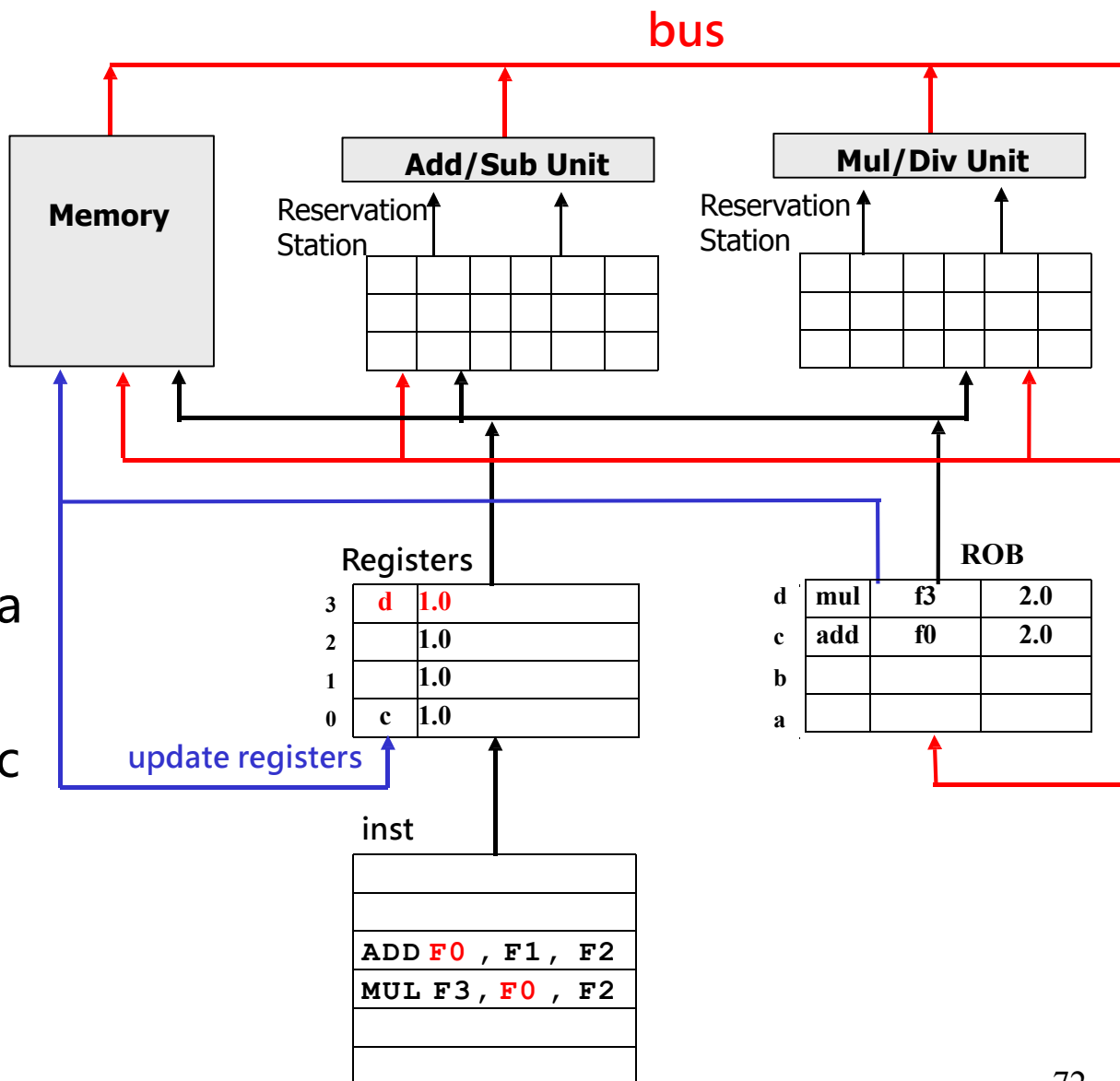
带ROB的乱序执行

DIV **F0**, F1, F2

MUL F3, F0, F2

ADD F0, F1, F2

MUL F3, F0, F2



div 发射, f1 和 f2 准备好

mul1 发射, f0 等待 ROB #a

add 发射, f1 和 f2 准备好

mul2 发射, f0 等待 ROB #c

add 写回

div 写回

div 提交, mul2 写回

mul 写回

mul1 提交

带ROB的乱序执行

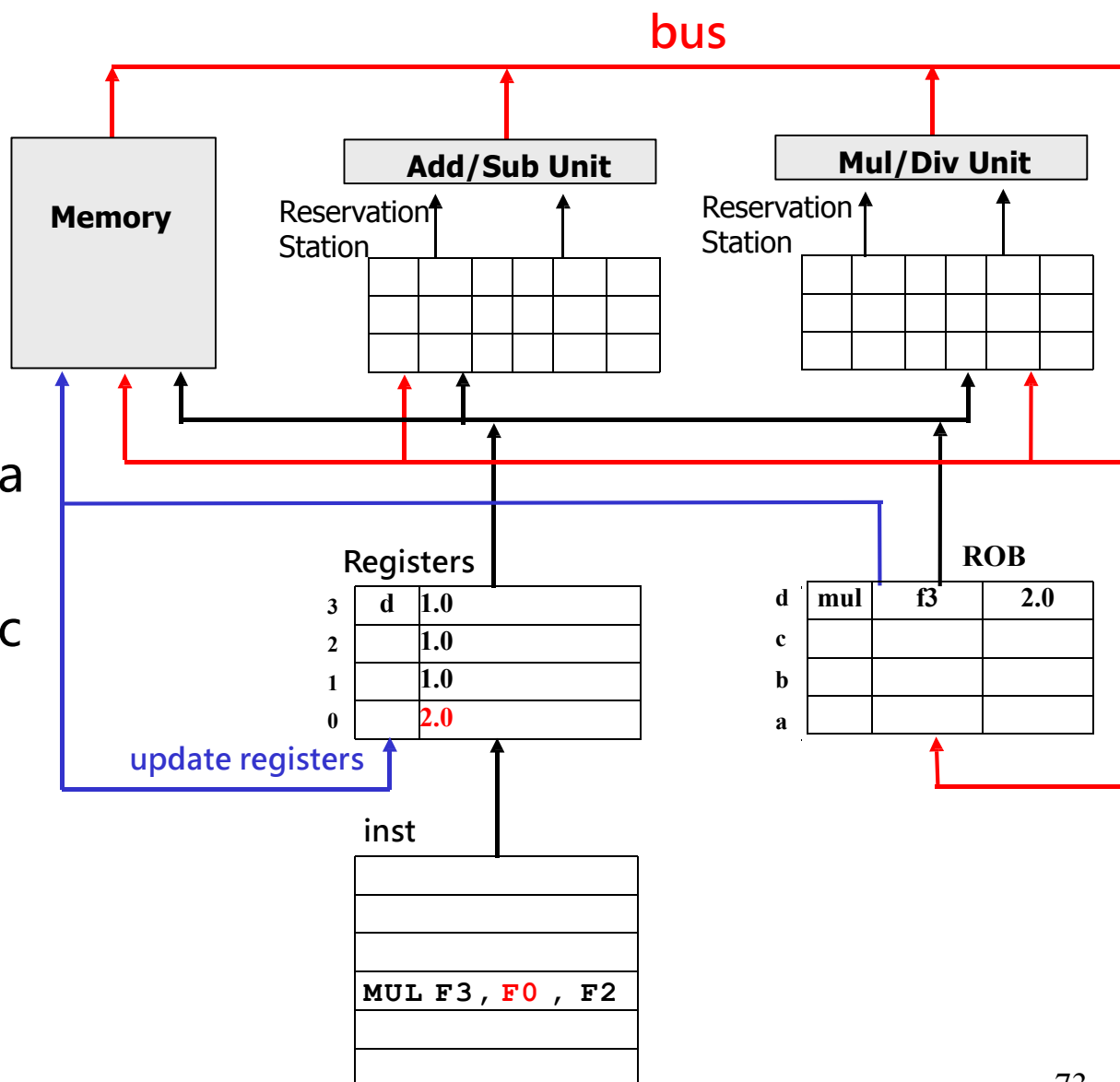
DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2

div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 ROB #a
add 发射, f1 和 f2 准备好
mul2 发射, f0 等待 ROB #c
add 写回
div 写回
div 提交, mul2 写回
mul 写回
mul1 提交
add 提交



带ROB的乱序执行

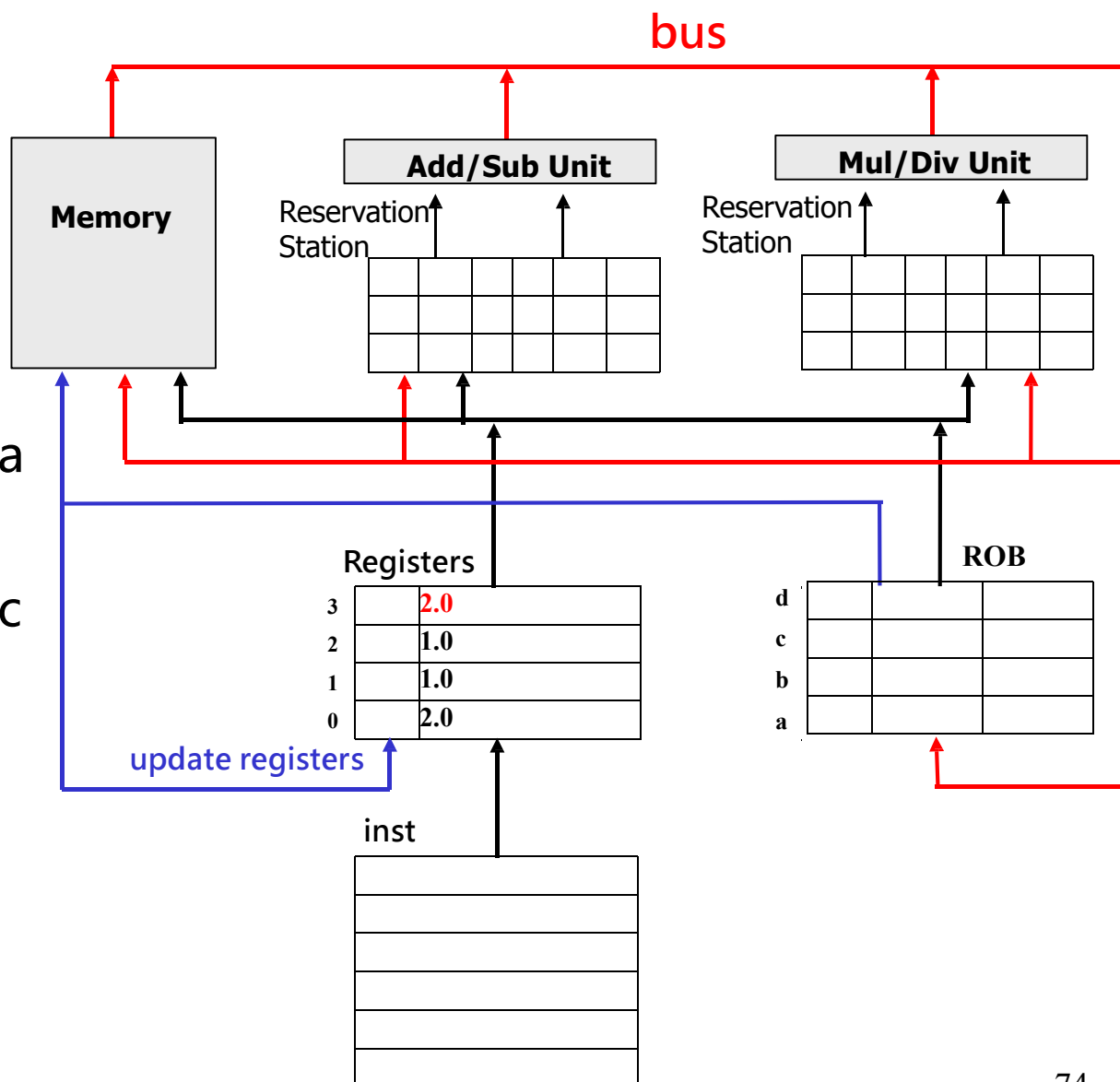
DIV **F0**, F1, F2

MUL F3, **F0**, F2

ADD **F0**, F1, F2

MUL F3, **F0**, F2

div 发射, f1 和 f2 准备好
mul1 发射, f0 等待 ROB #a
add 发射, f1 和 f2 准备好
mul2 发射, f0 等待 ROB #c
add 写回
div 写回
div 提交, mul2 写回
mul 写回
mul1 提交
add 提交
mul2 提交



带ROB的乱序执行

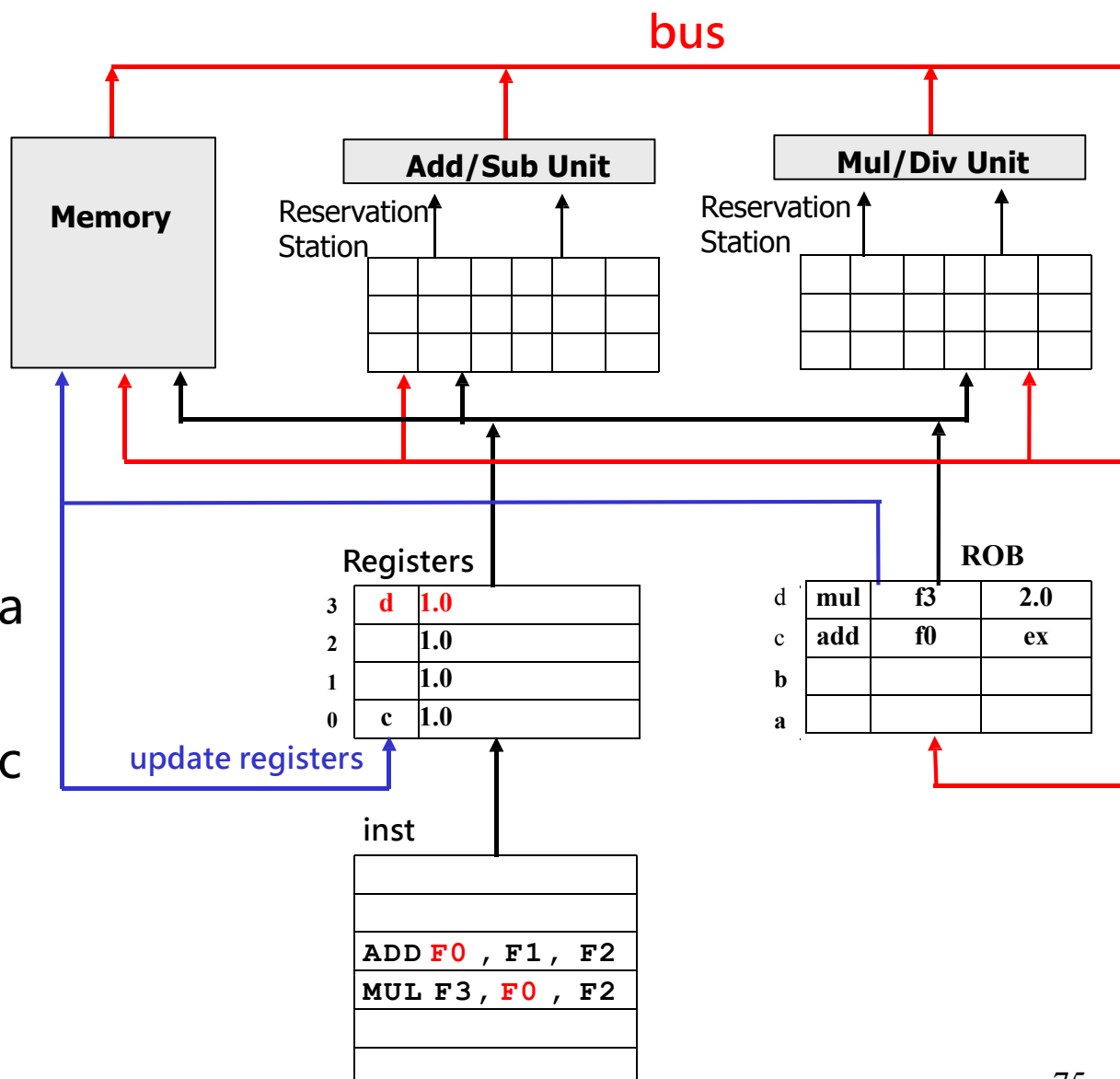
假设 ADD 发生异常

DIV F0, F1, F2

MUL F3, F0, F2

ADD F0, F1, F2

MUL F3, F0, F2



div 发射, f1 和 f2 准备好

mul1 发射, f0 等待 ROB #a

add 发射, f1 和 f2 准备好

mul2 发射, f0 等待 ROB #c

add 写回

div 写回

div 提交, mul2 写回

mul 写回

mul1 提交

带ROB的乱序执行

假设 ADD 发生异常

DIV F0, F1, F2

MUL F3, F0, F2

ADD F0, F1, F2

MUL F3, F0, F2

mul1 发射, f0 等待 ROB #a

add 发射, f1 和 f2 准备好

mul2 发射, f0 等待 ROB #c

add 写回

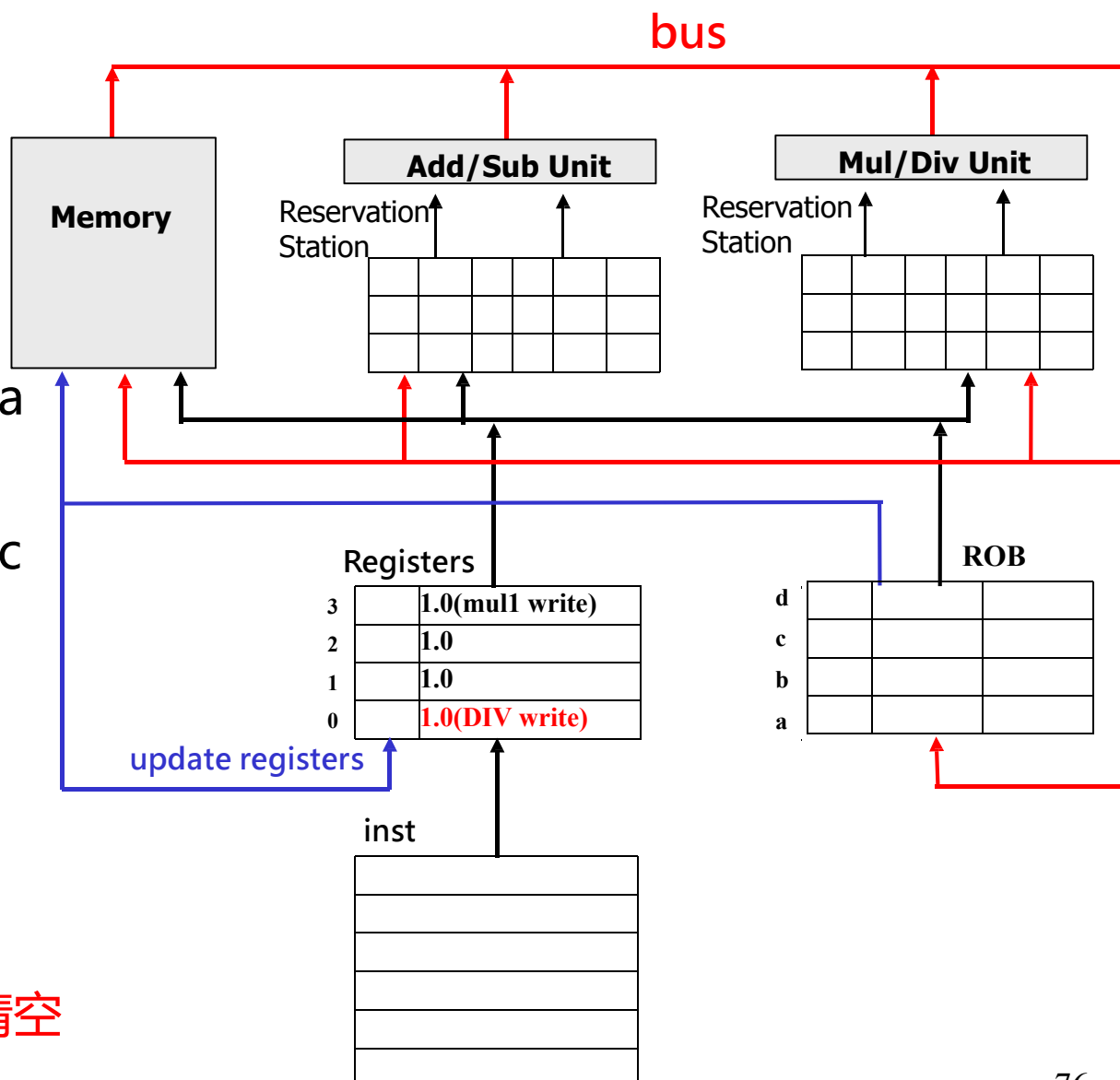
div 写回

div 提交, mul2 写回

mul 写回

mul1 提交

add 提交失败, 剩余指令清空



带ROB的乱序执行（总结）

- ❑ **Issue** : 发送指令到保留站(如果保留站和ROB都有空余), 用一个ROB表项存储结果; 读操作数 (检查是否ready)
- ❑ **Execute** : 如果所有操作数都已经ready, 执行指令, 否则监听总线, 如果发现操作数就读取, 并保存在保留站中
- ❑ **Write back** : 发送结果到总线, 释放保留站表项; ROB更新结果
- ❑ **Commit** : 如果前面指令都已经commit, 并且没有发生异常, 将结果从ROB写入register/memory (i.e., commit), 释放ROB表项; 如果前面指令发生异常, 清空指令列表和ROB等

访存操作

回顾: 相关性类型

- RAW (Read After Write)

mul r0 * r1 → **r2**

...

add **r2** + r3 → r4

- WAW (Write After Write)

mul r0 * r1 → **r2**

...

add r1 + r3 → **r2**

- WAR (Write After Read)

mul r0 * **r1** → r2

...

add r3 + r4 → **r1**

WAW & WAR 不是真相关，可以通过寄存器重命名消除

内存操作也存在相关性

如果 “r2” 和 “r3” 的值相同...

- RAW (Read After Write) – 真相关

st r1 → [r2]

...

ld [r3] → r4



- WAW (Write After Write)

st r1 → [r2]

...

st r4 → [r3]



- WAR (Write After Read)

ld [r2] → r1

...

st r4 → [r3]

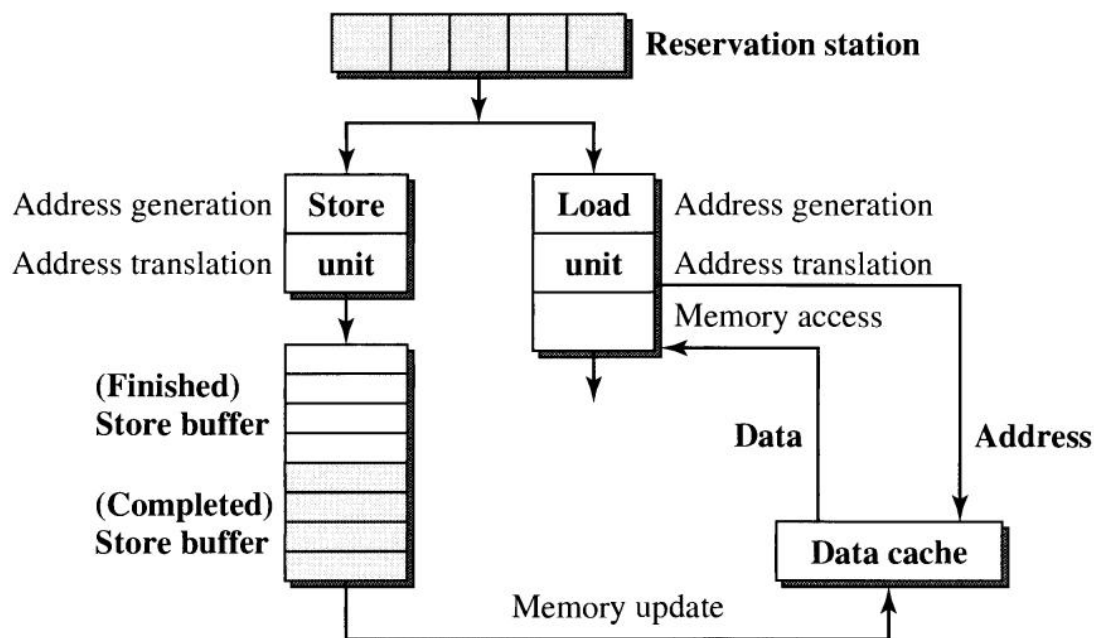


WAR/WAW不是真相关

- 但不能通过重命名消除，因为指令执行之前内存地址未知
- 需要用其他方法

内存操作一般流程

- 内存指令按照顺序进入保留站
- 从保留站按照指令顺序发射到对应单元
 - **load指令**：如果地址寄存器值准备好，就可以发送到load执行单元
 - **store指令**：需要等到地址寄存器值和要写入的数都准备好，才可以发送到store执行单元
 - **指令按照顺序发射**，前面的指令不发射，后面指令也不能发射



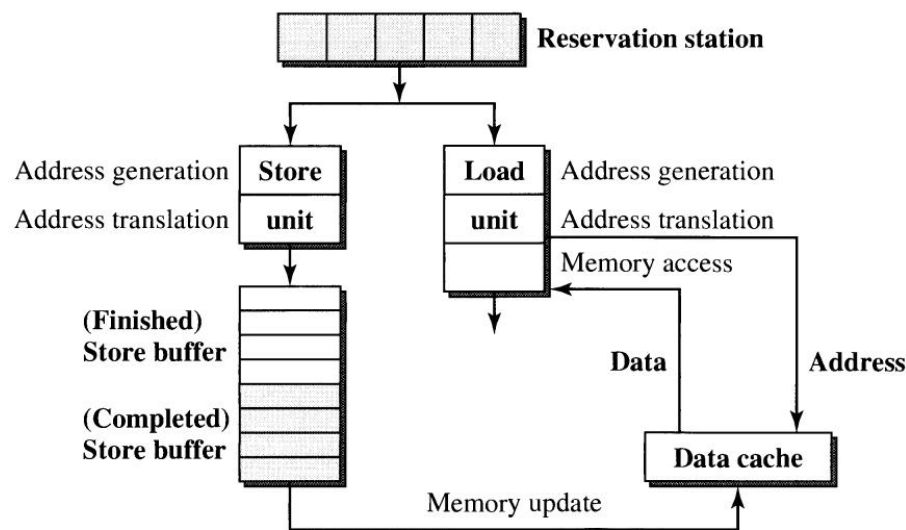
内存操作一般流程：load

□ load功能单元

- 以流水线方式完成三个步骤
 - 地址生成
 - 通常是寄存器+offset
 - 地址转换
 - 虚地址转换为物理地址
 - 内存访问(读数据)

□ load执行完三个步骤后

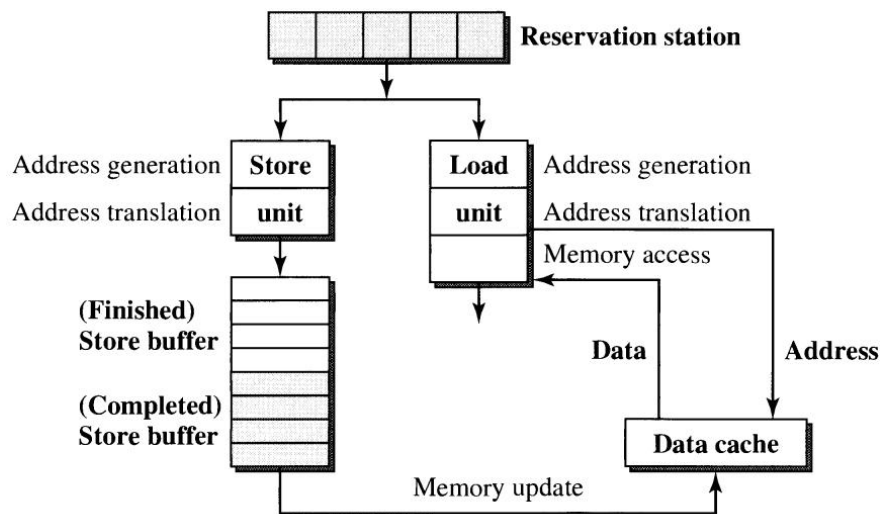
- 将读出来的数据缓存到ROB
- 指令最终提交时，再真正写寄存器，然后从ROB中删除指令
 - 如果cache命中，单周期可完成，否则需要等待(其它load指令不能前进)



内存操作一般流程: store

□ store功能单元

- 以流水线方式完成2个步骤
 - 地址生成
 - 通常是寄存器+ offset
 - 地址转换
 - 虚拟地址转换为物理地址



□ store的执行过程

- 执行完前2个阶段后(地址转换完成), 放入store buffer, 此时**指令状态为finished**
- 指令在ROB中完成提交后(按顺序), 从ROB中删除, 同时将store buffer中的**指令状态变为complete**
- store buffer中的指令真正写内存后, 指令从store buffer中删除, 该**指令最终完成(retired)**
- 异常发生时, store buffer中的状态为complete的指令仍然需要写入内存
 - 因为这些指令已经按照顺序在ROB中提交了

内存读写顺序

□ load和store指令可能读写同一个内存地址

- 引发RAW, WAR, WAW冲突
- 必须按照顺序执行，才能保证正确性

□ 保守方法是让所有load和store指令按照顺序执行

- 性能太差

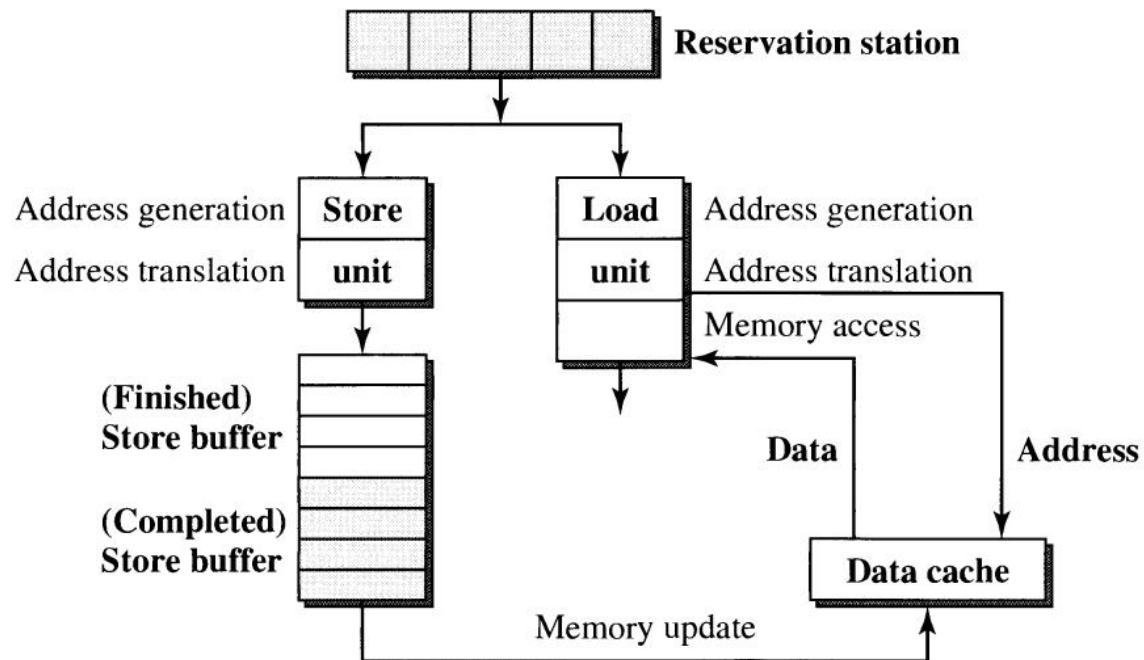
□ 解决方案->乱序执行：让load先于前面的store执行

- **load bypassing**: 如果load指令和前面的store指令地址不一样，load可以先执行
- **load forwarding**: 如果load指令和前面的store指令地址一样，load可以从前面的store（最近的一个）获取数据

内存读写顺序

□ load bypassing & load forwarding

- 内存地址计算完成后，检查store buffer是否有相同地址的store指令
- 如果没有，load继续执行(bypassing)
- 如果有，将相同地址中最近的那个store要写入的值forward给load



乱序发射?

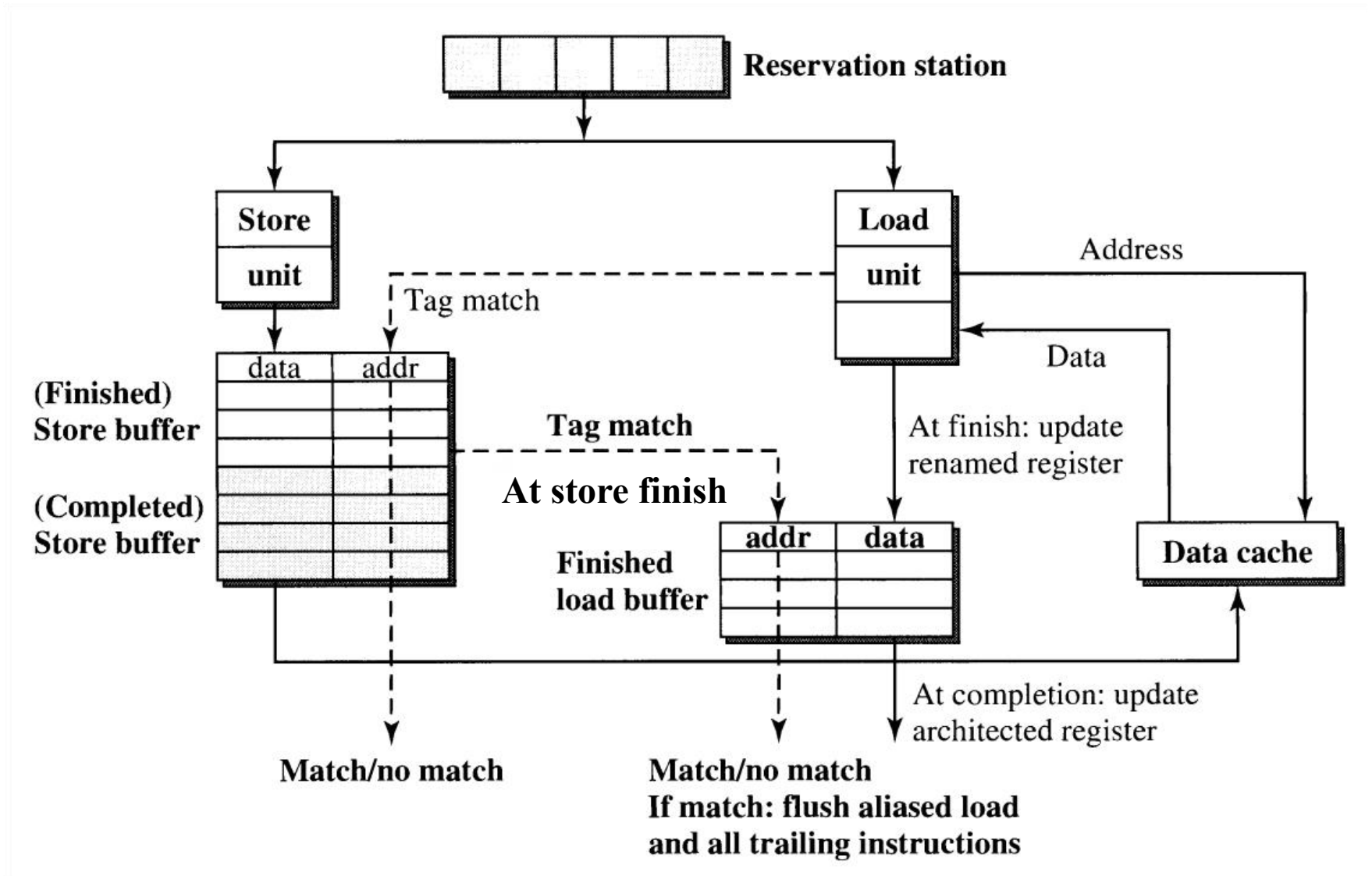
□ 前面都假设load和store指令按照顺序发射

- 如果前面的store操作数没准备好，后面的load指令无法发射
- **load发射时，排在后面的store肯定没有进入store buffer**

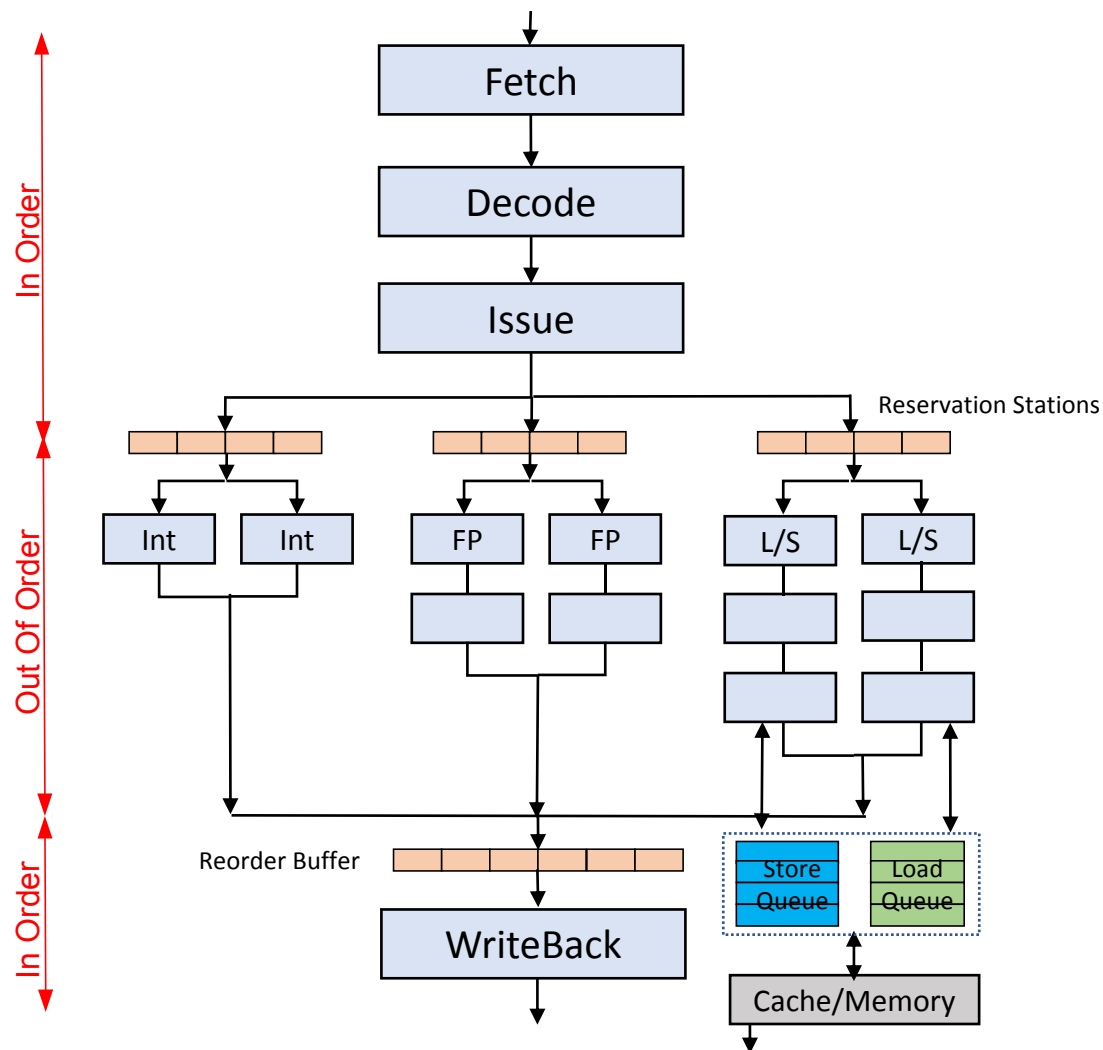
□ **乱序发射**：允许load指令先于前面的store指令发射

- load操作数准备好后就可以发射
- 增加**load buffer**，存放已经计算完地址、但还没有提交的load指令 (ROB还未提交)
- 每个store在进入store buffer时，检查load buffer中是否有相同地址的load
 - ✓ 如果没有，继续执行
 - ✓ 如果有，清除该load指令以及后续指令，并重新执行
 - 说明该load指令排在store后面，但先发射了，可能读取了错误的数
据(load进入load buffer时store还没有进入store buffer，没有发生
forward)

乱序发射?



截至目前，流水线：



- 单发射
- 乱序执行
- 有数据相关性
- 无分支指令
- 多周期指令
- Cache